

SANDIA REPORT

SAND2026-xxxx
Printed Date TBD



Sandia
National
Laboratories

Isogeometric Analysis (IGA) for Components

Chad B. Hovey, Jeremy L. Guthrie, David A. Najera-Flores,
Alejandro D. Carrizales, John M. Emery

Prepared by
Sandia National Laboratories
Albuquerque, New Mexico 87185
Livermore, California 94550

Issued by Sandia National Laboratories, operated for the United States Department of Energy by National Technology & Engineering Solutions of Sandia, LLC.

NOTICE: This report was prepared as an account of work sponsored by an agency of the United States Government. Neither the United States Government, nor any agency thereof, nor any of their employees, nor any of their contractors, subcontractors, or their employees, make any warranty, express or implied, or assume any legal liability or responsibility for the accuracy, completeness, or usefulness of any information, apparatus, product, or process disclosed, or represent that its use would not infringe privately owned rights. Reference herein to any specific commercial product, process, or service by trade name, trademark, manufacturer, or otherwise, does not necessarily constitute or imply its endorsement, recommendation, or favoring by the United States Government, any agency thereof, or any of their contractors or subcontractors. The views and opinions expressed herein do not necessarily state or reflect those of the United States Government, any agency thereof, or any of their contractors.

Printed in the United States of America. This report has been reproduced directly from the best available copy.

Available to DOE and DOE contractors from

U.S. Department of Energy
Office of Scientific and Technical Information
P.O. Box 62
Oak Ridge, TN 37831

Telephone: (865) 576-8401
Facsimile: (865) 576-5728
E-Mail: reports@osti.gov
Online ordering: <http://www.osti.gov/scitech>

Available to the public from

U.S. Department of Commerce
National Technical Information Service
5301 Shawnee Road
Alexandria, VA 22312

Telephone: (800) 553-6847
Facsimile: (703) 605-6900
E-Mail: orders@ntis.gov
Online order: <https://classic.ntis.gov/help/order-methods>



ABSTRACT

Isogeometric Analysis (IGA) promises **time savings** and **increased accuracy** relative to traditional Finite Element Analysis (FEA) approaches.

The savings arise in part from the elimination of the mesh generation step, which is often time-consuming and error-prone. We categorize this mesh step as **analyst time**. In addition, **compute time**, the second component of time savings, arises from faster convergence rates for IGA relative to traditional FEA. For a specific level of accuracy, IGA can often achieve the result with fewer degrees of freedom (DOF) and less computational effort [9, 10].

This report examines the performance premise of IGA through a series of case studies. It also discusses the challenges encountered during the implementation of a specific IGA workflow from Coreform, makers of the commercial software Coreform IGA (formerly Coreform Flex).

The primary goal of this report is to record our findings and raise awareness of IGA technology within the Sandia engineering analysis community.

Summary of Findings:

- We remain particularly interested in IGA, despite the challenges and limitations presented by the particular implementation, Coreform IGA, used for this study.
- We encourage integration of IGA technologies within the existing SNL Sierra framework, perhaps through use of Bézier extraction technologies.

This page intentionally left blank.

CONTENTS

1. Introduction	13
2. Background	15
2.1. Academia	16
2.2. Government Laboratories	17
2.2.1. Idaho National Laboratory and MOOSE	17
2.2.2. Other Government Laboratory Contributions	18
3. Models	21
3.1. C-Frame	21
3.2. Door Hinge	26
3.3. Circuit Board	26
4. Discussion	27
4.1. Workflow and Learning Curve	27
4.2. Software Stability and Input Deck Compatibility	27
4.3. Distinguishing IGA as a Technology from Coreform as a Platform	28
4.4. Path Forward for IGA at Sandia	28
5. Conclusion	31
References	33
Appendices	35
A. Basis Functions	35
A.1. Lagrange Basis Functions	35
A.2. Legendre (GLL) Basis Functions	36
A.3. Bézier (Bernstein) Basis Functions	37
B. Bézier Extraction	43
C. IGA Input Files	45
C.1. C-Frame	45
C.1.1. jobs_coarse.iga	45
C.1.2. jobs_fine.iga	46
C.1.3. jobs_fine_x2.iga	48
C.2. Door Hinge	49
C.2.1. door_hinge_failure_workflow.txt	49

C.2.2.	<code>job_door_hinge_failure.iga</code>	52
C.2.3.	<code>read_force_vm.py</code>	56
C.3.	Circuit Board	58
C.3.1.	<code>cb_modal_workflow_journal_separate.py</code>	58
C.3.2.	<code>cb_modal_workflow_journal.py</code>	61
C.3.3.	<code>cb_workflow_journal.py</code>	63

DRAFT

LIST OF FIGURES

Figure 2-1. The time distribution for a typical engineering analysis workflow, reproduced from [9] and the original studies by Blacker et al. at Sandia National Laboratories. The chart illustrates that approximately 80% of total analyst time is consumed by geometry preparation and mesh generation, while only 20% is spent on the actual numerical solution and results refinement.	19
Figure 3-1. C-frame geometry.	21
Figure 3-2. Fine mesh models for C-frame geometry. (a,b) IGA model. (c,d) Tetrahedral FEA model. (e,f) Hexahedral FEA model.	22
Figure 3-3. Mode shapes and frequencies for the C-frame IGA model. The first six modes are shown. The IGA model used is the first refinement model (13,332 DOF).	23
Figure 3-4. Model frequency convergence as a function of degree of freedom (DOF) for IGA and FEA models. FEA included SSD tetrahedral (10-noded) and SSD hexahedral (8-noded) meshes. DOF count increases from left (coarse) to right (fine). The IGA model shows faster convergence with DOF compared to the traditional FEA models. DOF count specifics appear in Table 3-1.	24
Figure 3-5. Processor time versus degree of freedom (DOF) for C-frame IGA and FEA models. The IGA model shows lower processor time for a given DOF count compared to the traditional FEA models.	25
Figure 3-6. Door hinge geometry used for nonlinear static analysis.	26
Figure 3-7. Circuit board geometry used for nonlinear static and modal analysis.	26
Figure A-1. Lagrange basis functions for 2-, 3-, and 4-noded 1D element.	39
Figure A-2. Legendre basis functions for 2-, 3-, and 4-noded 1D element.	40
Figure A-3. Bézier basis functions for 2-, 3-, and 4-noded 1D element.	41

This page intentionally left blank.

LIST OF TABLES

Table 3-1. C-frame model resolutions (DOF #), run times, and processors.	25
---	----

DRAFT

This page intentionally left blank.

ABBREVIATIONS AND ACRONYMS

- CAD - Computer-Aided Design
- GitLab repository - *cee-gitlab.sandia.gov/structMechTools/iga*
- FEA - Finite Element Analysis
- IGA - Isogeometric Analysis
- NURBS - Non-Uniform Rational B-Splines
- SSM - Sierra Solid Mechanics

DRAFT

This page intentionally left blank.

1. INTRODUCTION

Traditional Finite Element Analysis (FEA) has been a standard for computational mechanics for decades. Its ecosystem is mature, with extensive validation and a wide range of element formulations.

However, FEA suffers from one large inherent limitation: The geometry must be meshed, and the traditional FEA mesh only approximates the geometry when it has a CAD origin. The mesh generation process introduces geometric approximation errors, requires significant preprocessing time, and (depending on the formulation) creates C^0 -continuous approximations that may be inadequate for higher-order problems. Design optimization workflows require repeated geometry-to-mesh conversions. Each conversion consumes valuable analyst time and risks losing geometric fidelity.

Introduced by Hughes *et al.* in 2005 [9], IGA proposes to address these drawbacks through its use of the **same** basis for geometry and analysis, effectively eliminating the mesh step. The geometric basis for CAD is often Non-Uniform Rational B-Splines (NURBS), which arise from Bézier (Bernstein) basis functions.

In contrast, traditional FEA uses Lagrange (or, alternatively Legendre) interpolations as basis functions for analysis (see Appendix A). In instances where the geometry has a CAD description and that description is NURBS-based, the traditional FEA mesh will only approximate the original geometry. As a result, the FEA analysis will only approximate the true physics of the system.

IGA maintains exact geometric representation at every refinement level. This eliminates geometric discretization error and removes the mesh generation step entirely. Spline basis functions also carry higher-order continuity by nature (C^1 , C^2 , or arbitrary C^k). That continuity gives better accuracy per degree of freedom, especially for fourth-order problems and problems that need smooth solution fields.

IGA integrates directly with CAD systems, so analysts can work on design geometries without a separate meshing step and iterate faster on shape optimization. It also captures complex phenomena, including boundary layers, stress concentrations, and wave propagation, using coarser discretizations than traditional FEA needs.

Despite its advantages, IGA faces several challenges that have tempered its widespread adoption. The software ecosystem is still nascent, relative to traditional FEA. Most commercial and laboratory tools are built around mature FEA platforms, and finding IGA-native implementations can be difficult.

Additional hurdles stem from the additional mathematical machinery that must be created, to include surface trimming, boolean operations, and complex multi-patch CAD geometry

management. Even something as fundamental as defining the volume for a 3D solid can get tedious when the topology gets complicated.

While IGA is generally known to require fewer degrees of freedom than FEA for a given level of accuracy, IGA can actually become *more* expensive than FEA *per degree of freedom* because of the wider support of the basis functions and the heavier math required for quadrature. That said, the *overall processor time* for the IGA models still promises to be lower than that for the FEA models (a result corroborated by our limited case study, see Table 3-1).

Finally, one of the biggest barriers to adoption is likely human in nature. Engineers have spent decades developing FEA workflows and creating large validation databases; they have become very proficient with FEA tools they already know and trust. Adoption of a new method not only requires retooling of the analysis codes, it requires recreation of the analysts' processes.

Nonetheless, adoption of IGA has outpaced FEA. According to Hughes [10] (at 12:24/1:02:22), "The growth rate of isogeometric analysis is much faster than for finite element analysis, which seemed to sweep away the world, but it took a long time for finite element analysis to do that, depending on where you place the start, say the 1950s or 1960s."

DRAFT

2. BACKGROUND

Within industry, several commercial software packages now support IGA capabilities. Examples include Ansys, Siemens NX Nastran, Dassault Systèmes Abaqus, Altair HyperWorks, and Coreform IGA (formerly Coreform Flex). These packages vary widely in maturity, from basic NURBS support to full multi-patch spline workflows.

Ansys (via LS-DYNA) is currently the most prominent large-scale commercial software vendor of IGA. Since the acquisition of LSTC, Ansys has integrated IGA capabilities into LS-DYNA, making it the most mature commercial solver for IGA applications.

LS-DYNA supports NURBS-based shells and solids, with specific emphasis on crash analysis and large-deformation problems. The first NURBS-based shell formulations appeared in LS-DYNA in 2011. Thereafter, IGA capabilities were expanded to solid elements, implicit and explicit analysis, contact, trimmed NURBS shells, and unstructured boundary-fitted approaches using Bézier extraction. In late 2025 and early 2026, Ansys expanded its “Trimmed IGA Solids” capabilities, which allow for immersed boundary analysis wherein the physics is solved on a background grid that respects the CAD geometry via trimming operations. No traditional finite element mesh is required [15].

One example of IGA performance on thin-walled structures with complex surface geometry was provided by Novozhilov and Leidinger [15]. Their use of the trimmed IGA approach permitted larger time steps and required fewer elements to achieve similar accuracy compared to traditional FEA approaches. Specifically, their IGA model delivered comparable accuracy to a TET-16 formulation, while reducing compute time by a factor of $21\times$.

Siemens Digital Industries Software (NX and Simcenter Nastran) has added IGA capabilities to its platform. Siemens offers NURBS-based shell and solid elements, with capabilities for multi-patch coupling and adaptive refinement. These implementations handle static, dynamic, and thermal structural analysis. As of June 2025 and continuing into 2026, Siemens moved beyond previews of NX Performance Predictor. While not branded as IGA, the underlying technology leverages spline-based representations to enhance accuracy and efficiency in performance prediction workflows. Siemens also has integrated subdivision modeling (an alternative to NURBS) that maintains higher-order continuity (C^2) [17].

Dassault Systèmes Abaqus has been historically more conservative with native IGA integration. Abaqus supports IGA through high-fidelity toolkits and user-defined elements (UEL).

Altair HyperWorks has prototyped IGA capabilities through research collaborations. It has not yet integrated IGA into its mainstream offerings. The 2026 release of HyperWorks included IGA-like capabilities through its “warp-map” tools [1].

Coreform is a computational mechanics software company focused on isogeometric analysis. Coreform’s flagship product is Coreform Cubit, a powerful meshing and geometry processing tool that supports IGA workflows. Coreform also offers Coreform IGA (formerly Coreform Flex), a solver specifically designed for isogeometric analysis applications. Michael Scott is the CEO of Coreform, Matt Sederberg is the Chief Strategic Officer, Greg Vernon is the Director, and Thomas J.R. Hughes is the Senior Advisor [4]. The authors of this SAND report worked periodically with Sederberg and Vernon during Q1 of FY2026 to obtain access to Coreform IGA and to discuss IGA capabilities and workflows.

2.1. Academia

The academic community has created several open-source software frameworks for IGA. Three of the most widely used are G+Smo, GeoPDEs, and PetIGA.

G+Smo (Geometry + Simulation Modules) (pronounced “gismo”) is a comprehensive, open-source C++ library for IGA [12]. Development of the library was initiated in the summer of 2012, coinciding with the kick-off of the Austrian National Research Network “Geometry + Simulation,” funded by the Austrian Science Fund (FWF) [7]. The library is jointly developed by contributors at Johannes Kepler University Linz (JKU Linz) and at the RICAM Institute of the Austrian Academy of Sciences. G+Smo is an object-oriented, cross-platform, template C++ library that implements a generic concept for IGA, based on abstract classes for geometry maps, discretization bases, assemblers, and solvers.

G+Smo’s code is dimension-independent, implemented via template metaprogramming. As a result, the same implementation applies to curves, surfaces, and volumes. The library supports B-spline, NURBS, hierarchical B-spline, and truncated hierarchical B-spline bases of arbitrary polynomial order. The library is licensed under the Mozilla Public License v2.0 (MPL 2.0), a *copyleft* license that permits both open and proprietary use of derived software provided that modifications to the library itself are shared back under the same license. Annual developer workshops—most recently at the University of West Bohemia, Pilsen, Czech Republic in January 2026—demonstrate the library’s active and growing community [7].

GeoPDEs is an open-source framework for IGA, written in GNU Octave and compatible with MATLAB [6, 18]. Originally released in 2010, the package seeks to provide a common and flexible environment for testing isogeometric approaches to solutions for PDEs. The same code handles curves, surfaces, and volumes, so GeoPDEs is dimension-independent by construction. It also supports multi-patch domains with conforming interfaces, divergence- and curl-conforming spline spaces, and adaptive refinement using hierarchical B-splines. The package ships with example problems spanning Poisson, linear elasticity, advection-diffusion, Stokes, Navier-Stokes, Maxwell, and Kirchhoff-Love shell equations. GeoPDEs is released under the GNU General Public License v3. It was developed principally by researchers at IMATI-CNR (Pavia, Italy) and the University of Santiago de Compostela (Spain), among other contributors, and remains actively maintained.

PetIGA is an open-source framework targeting high-performance, large-scale IGA computations [5]. PetIGA is built on top of PETSc (the Portable, Extensible Toolkit for Scientific

Computation), a widely used library for scalable solution of partial differential equations. This design separates parallelism from algorithm choice. Users get one platform for both prototyping and large-scale production runs. PetIGA can assemble matrices and vectors from a Galerkin weak form discretized with NURBS basis functions, and has been demonstrated to scale to 4,096 parallel cores. Applications demonstrated within the PetIGA framework include hyperelasticity, the Cahn-Hilliard phase-field equation, turbulent Navier-Stokes using variational multiscale methods, and Reissner-Mindlin shell formulations. PetIGA was developed with support from the King Abdullah University of Science and Technology (KAUST) Extreme Computing Research Center [5].

2.2. Government Laboratories

Government laboratories have made significant investments in IGA technology, both through the development of internal simulation frameworks and through partnerships with commercial software vendors.

2.2.1. *Idaho National Laboratory and MOOSE*

Idaho National Laboratory (INL) developed the Multiphysics Object-Oriented Simulation Environment (MOOSE), a general-purpose, open-source, finite-element-based framework for solving systems of coupled partial differential equations on high-performance computing platforms [13]. Initial development began in 2004–2005 with funding from INL’s Laboratory Directed Research and Development (LDRD) program, and the framework has since accumulated more than 60 known MOOSE-based application codes and over 70,000 framework package downloads [13].

Starting in 2021, INL partnered with Coreform LLC to extend MOOSE with isogeometric analysis capabilities through a series of U.S. Department of Energy Small Business Innovation Research (SBIR) awards [11]. The collaboration introduced Coreform’s spline-based meshing technology into the MOOSE workflow, providing researchers with a free-to-use, robust tool for generating high-quality, geometry-conforming meshes for MOOSE simulations. A motivating application was the simulation of nuclear fuel pellets pressing against protective rod cladding, where smooth, geometry-accurate contact pressure calculations are critical; the spline-based representation improved both accuracy and computational efficiency relative to traditional rectangular meshing approaches [11].

Building on this initial collaboration, Coreform completed a Phase I SBIR that demonstrated a proof-of-concept integration between the MOOSE nuclear simulation code and Coreform Cubit, along with enhancements enabling full IGA spline meshes to be read and used directly within MOOSE simulations [3]. As part of this work, Coreform released Coreform Cubit Learn, a free non-commercial license of Coreform Cubit, making the same meshing toolset used in industry available to academic and government researchers. A subsequent Phase II SBIR award of \$1.1 million was made to Coreform to continue this work [3]. The

Phase II effort aims to combine Coreform’s prior IGA-for-MOOSE and MFEM-IGA integration projects to enable IGA simulations within MOOSE that can run on graphics processing units (GPUs), which is expected to significantly accelerate computation for nuclear energy engineering workflows.

MOOSE’s IGA capabilities are documented within the framework itself. A canonical example in the MOOSE documentation demonstrates IGA applied to a C-frame structural analysis problem—the same geometry studied in this report—illustrating the workflow from Coreform Cubit spline mesh generation through MOOSE solid mechanics analysis [14].

2.2.2. *Other Government Laboratory Contributions*

The Sandia National Laboratories analysis of the engineering design-to-analysis workflow—documented by Ted Blacker, Manager of Simulation Sciences at Sandia—is an oft-cited motivation for IGA. The study found that at Sandia, geometry preparation consumes approximately 57% of overall analysis time and mesh generation an additional 20%, leaving only 23% for actual analysis work [16]. This approximate 80/20 modeling-to-analysis ratio is recognized as a persistent bottleneck that IGA technology directly addresses.

The figure from Blacker’s study is shown in Figure 2-1. The figure illustrates the time distribution for a typical engineering analysis workflow, highlighting the disproportionate amount of time spent on geometry preparation and mesh generation compared to the actual analysis phase. This data underscores the potential for IGA to significantly reduce overall analysis time by eliminating the need for separate geometry and mesh preparation steps.

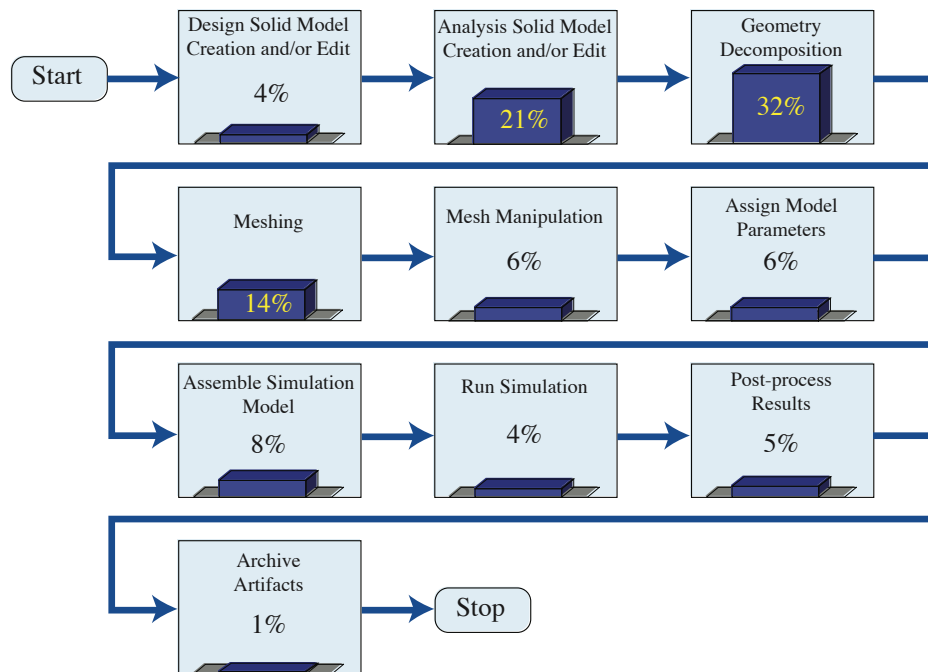


Figure 1.2 Estimation of the relative time costs of each component of the model generation and analysis process at Sandia National Laboratories. Note that the process of building the model completely dominates the time spent performing analysis. (Courtesy of Michael Hardwick and Robert Clay, Sandia National Laboratories.).

Figure 2-1. The time distribution for a typical engineering analysis workflow, reproduced from [9] and the original studies by Blacker et al. at Sandia National Laboratories. The chart illustrates that approximately 80% of total analyst time is consumed by geometry preparation and mesh generation, while only 20% is spent on the actual numerical solution and results refinement.

This page intentionally left blank.

3. MODELS

We explored several models, three of which have been chosen for inclusion into this report. The first is the C-Frame (Figure 3-1), which is a common test case for IGA and FEA comparisons. The second is the door hinge (see Figure 3-6). The third is a circuit board (Figure 3-7).

3.1. C-Frame

We initially used the C-Frame model, shown in Figure 3-1, as a test case to explore IGA workflows and to compare against traditional FEA approaches. The C-Frame geometry is included in the Coreform IGA distribution; no additional cleanup was needed for IGA.

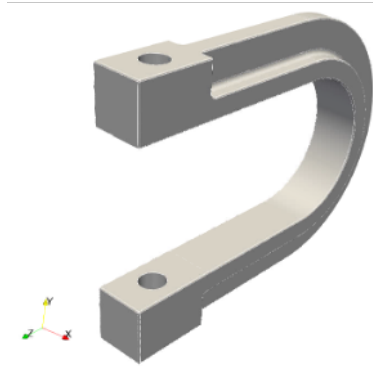
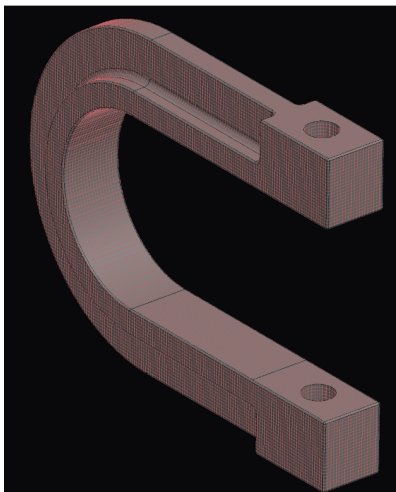


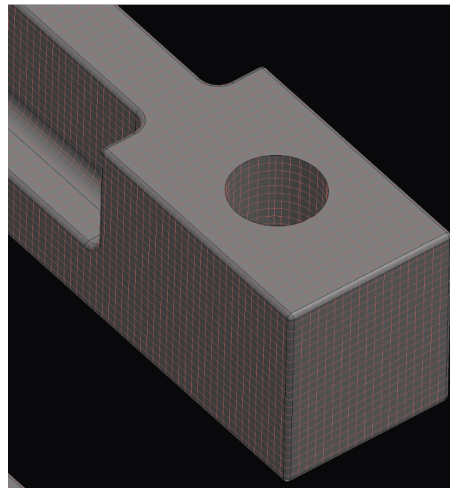
Figure 3-1. C-frame geometry.

We performed a modal analysis on the C-Frame geometry using both IGA and traditional FEA approaches, with the goal of comparing convergence rates and computational efficiency between the two methods. The IGA model was generated using Coreform Cubit, while the traditional FEA models were generated with Sandia’s Cubit, using standard meshing techniques.

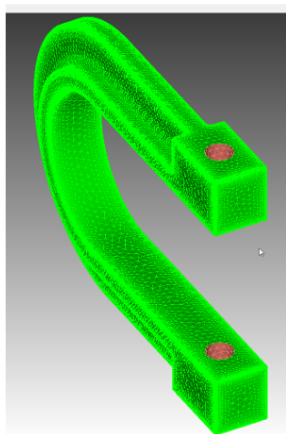
The modal analysis was performed using the Sierra Solid Dynamics (SSD) solver, with a direct solver for the eigenvalue problem. We requested modes in the frequency range of (1, 4000) Hz to capture the relevant dynamic behavior of the structure. The IGA analyses were solved locally on a laptop with 1 CPU, Windows 11, 32 GB RAM, and an Intel i7 (13th gen) processor. The SSD analyses were run with multiple cores on the Sandia HPC. The results of the modal analysis, including mode shapes and frequencies, were compared between the IGA and traditional FEA models to assess the performance benefits of IGA in terms of accuracy and computational efficiency.



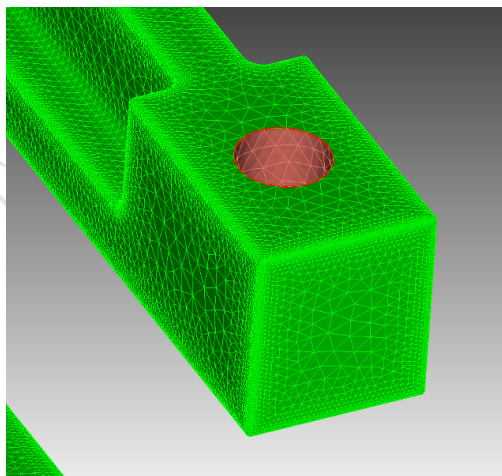
(a) IGA model with 383,250 DOF.



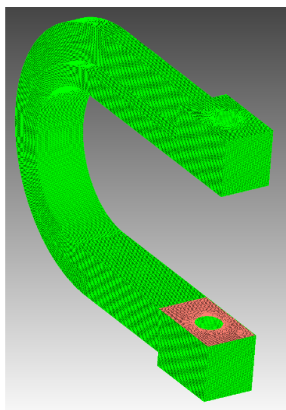
(b) IGA model zoomed in.



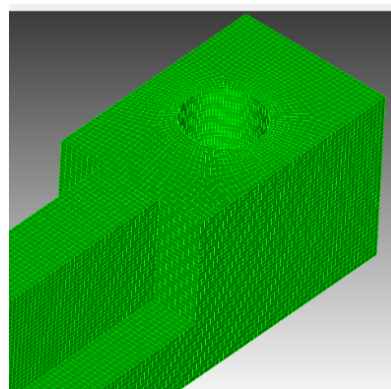
(c) Tetrahedral FEA model with 4,072,959 DOF.



(d) Tetrahedral FEA model zoomed in.



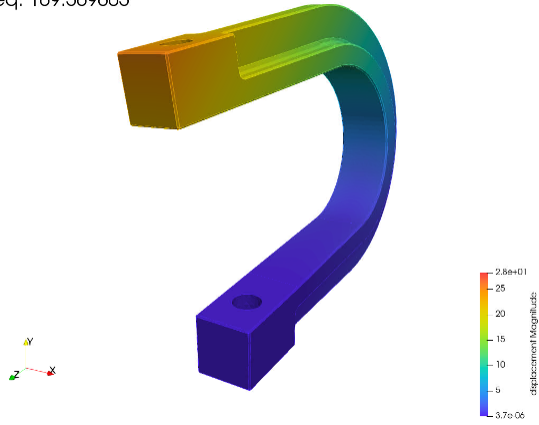
(e) Hexahedral FEA model with 1,167,897 DOF.



(f) Hexahedral FEA model zoomed in.

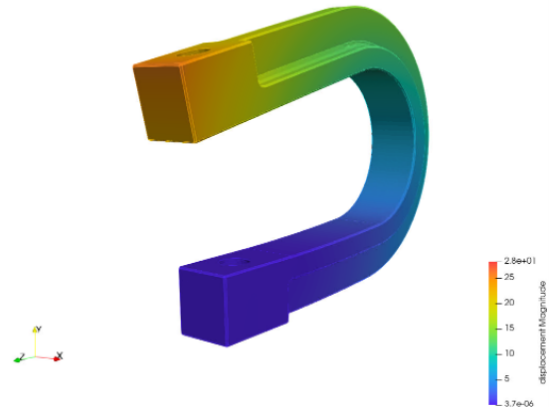
Figure 3-2. Fine mesh models for C-frame geometry. (a,b) IGA model. (c,d) Tetrahedral FEA model. (e,f) Hexahedral FEA model.

Freq: 169.569605



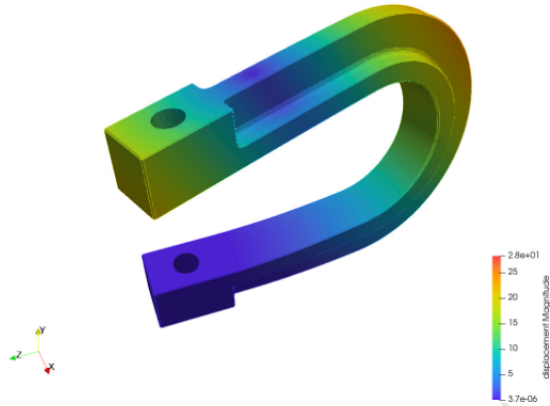
(a) Mode 1

Freq: 301.280628



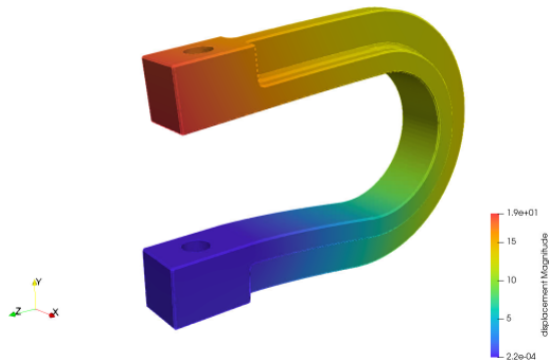
(b) Mode 2

Freq: 409.101071



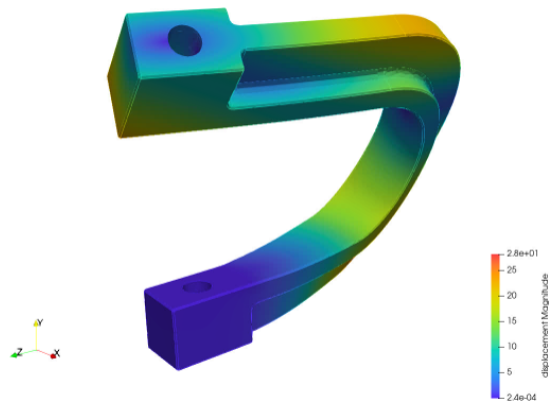
(c) Mode 3

Freq: 536.189570



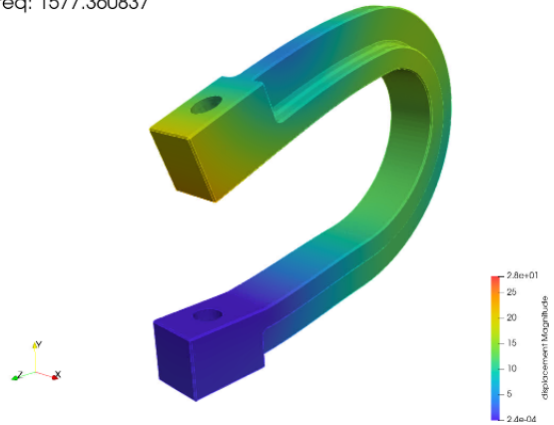
(d) Mode 4

Freq: 1221.273218



(e) Mode 5

Freq: 1577.360837



(f) Mode 6

Figure 3-3. Mode shapes and frequencies for the C-frame IGA model. The first six modes are shown. The IGA model used is the first refinement model (13,332 DOF).

The model frequency convergence results are shown in Figure 3-4. The IGA model shows faster convergence with mesh refinement compared to the traditional FEA models. The FEA models included solid standard SSD tetrahedral (10-noded) and hexahedral (8-noded) elements. The DOF count increases from left (coarse) to right (fine). The DOF count specifics appear in Table 3-1. The source for Figure 3-4 is the Python script *read_modes_upd.py* and the Excel file *cframe_modal_convergence.xlsx* in the *pre_post_processing/cframe* folder on the GitLab repository.

The data in Table 3-1 summarizes the model resolutions (DOF counts), run times, and number of processors used for each C-frame model. The IGA models were run on a single core, while the FEA models were run on multiple cores to reduce wall time. The cores were multiplied by wall time to estimate total processor time for the FEA models. These comparisons are shown in Figure 3-5. The source for Figure 3-5 is the Python script *processor_time_v_dof_plot.py* in the *pre_post_processing/cframe* folder on the GitLab repository.

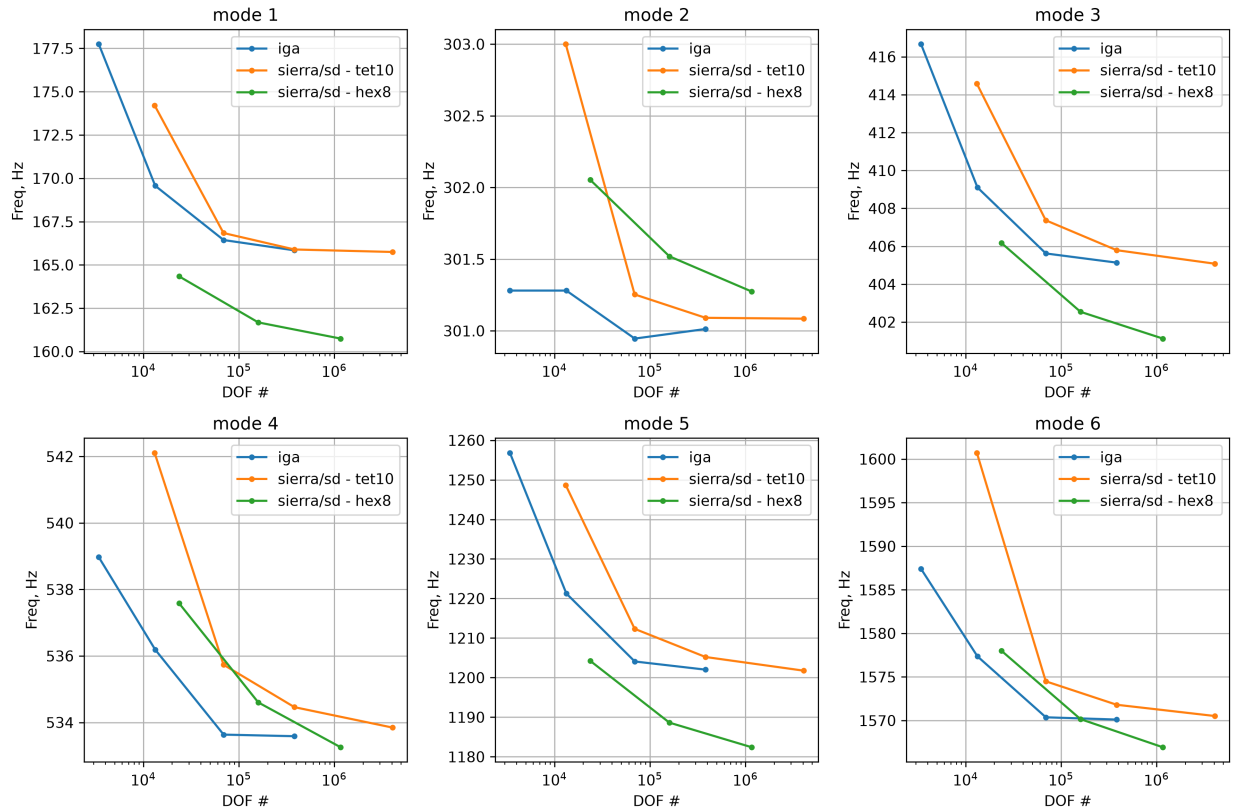


Figure 3-4. Model frequency convergence as a function of degree of freedom (DOF) for IGA and FEA models. FEA included SSD tetrahedral (10-noded) and SSD hexahedral (8-noded) meshes. DOF count increases from left (coarse) to right (fine). The IGA model shows faster convergence with DOF compared to the traditional FEA models. DOF count specifics appear in Table 3-1.

Table 3-1. C-frame model resolutions (DOF #), run times, and processors.

Analysis Type	Coarse	← →	Fine
IGA	3,420	13,332	69,426
	15s (1 core)	37s (1 core)	209s (1 core)
SSD tet10	13,146	69,234	382,791
	116s (128 cores)	141s (128 cores)	30s (128 cores)
SSD hex8	23,682	160,059	1,167,897
	6s (12 cores)	20s (128 cores)	75s (128 cores)
			—

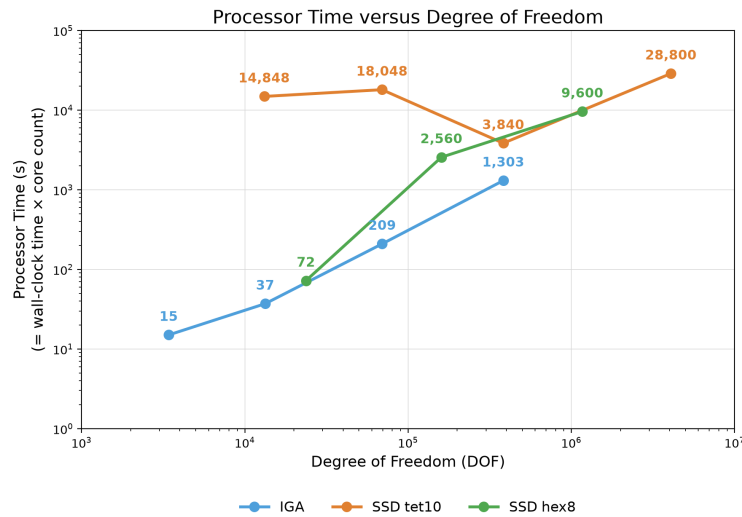


Figure 3-5. Processor time versus degree of freedom (DOF) for C-frame IGA and FEA models. The IGA model shows lower processor time for a given DOF count compared to the traditional FEA models.

The C-Frame results establish a performance baseline for IGA relative to traditional FEA approaches at Sandia. For the subsequent models—the door hinge and circuit board—we elected not to repeat the full multi-resolution convergence study across all three discretization strategies (IGA, SSD tet10, and SSD hex8). Generating and running the complete suite of coarse-to-fine models for each geometry requires substantial analyst and compute time. We had already characterized convergence behavior on the C-Frame, so we judged a full replication was not worth the added cost. We expect the door hinge and circuit board to follow the same trend: IGA reaches comparable accuracy at lower DOF counts. We have not verified that expectation for these specific geometries and loading conditions.

3.2. Door Hinge

The door hinge model was set up for nonlinear static analysis, with features including frictional contact and plasticity. The geometry was obtained from a CAD model, and no additional cleanup was needed for IGA. The model was analyzed using the Sierra Solid Mechanics (SSM) solver, with appropriate material models and contact definitions to capture the nonlinear behavior of the hinge under loading.

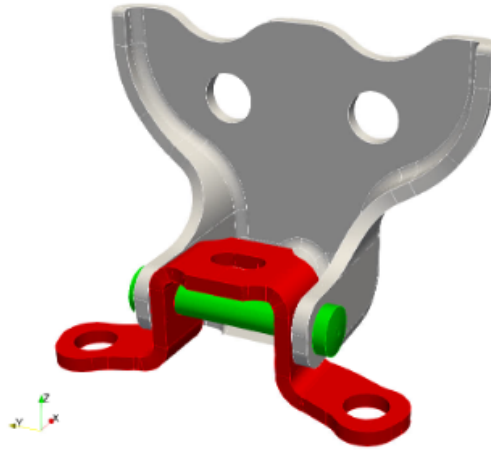


Figure 3-6. Door hinge geometry used for nonlinear static analysis.

3.3. Circuit Board

The circuit board model was set up for both nonlinear static and modal analysis, with features including linear elastic materials. The geometry was obtained from a CAD model, and required slight cleanup for IGA. The nonlinear static analysis was performed using the Sierra Solid Mechanics (SSM) solver, while the modal analysis was conducted to extract the first six modes of vibration.

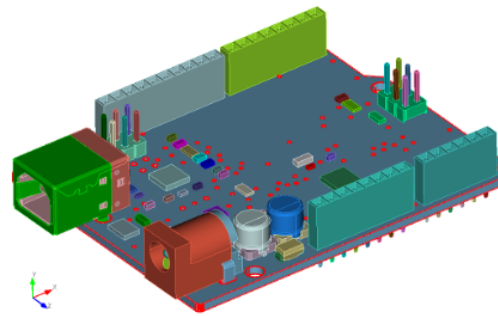


Figure 3-7. Circuit board geometry used for nonlinear static and modal analysis.

4. DISCUSSION

In a nutshell, IGA proved faster and more accurate than FEA. However, the analyst time devoted to getting the Coreform IGA workflow up and running was substantial, and the software stability and post-processing workflows were sources of friction that limited our enthusiasm for the Coreform offering. The following sections discuss these assessments in more detail.

4.1. Workflow and Learning Curve

We spent a substantial fraction of project time on the IGA learning curve, not on producing analysis results. Traditional FEA workflows have decades of documentation, institutional knowledge, and internal training behind them at Sandia. IGA workflows remain unfamiliar to Sandia’s engineering analysis community. The analyst must learn each stage of the pipeline from scratch: CAD geometry, spline mesh generation in Coreform Cubit, solution in Coreform IGA, and post-processing. This startup cost is real and should factor into any institutional assessment of IGA adoption.

We also had trouble getting computational access to the Coreform IGA license server through Sandia’s firewall. License provisioning and network configuration issues introduced delays early in the project, reducing the window available for productive analysis. These are not problems with IGA as a technology. They are typical of adopting any early-stage external software, and teams doing similar work should expect them.

4.2. Software Stability and Input Deck Compatibility

A recurring frustration encountered during this project was the incompatibility of previously working Coreform input decks with updated versions of the software. Input decks that executed successfully under an initially-used version failed to run after subsequent software updates, requiring rework and re-validation of existing analyses.

For reproducibility, the input decks used in this study are archived in the appendix of this report and on the project GitLab repository at [cee-gitlab.sandia.gov/structMechTools/iga](https://gitlab.sandia.gov/structMechTools/iga). Teams building on this work should treat input deck compatibility as a version-management concern and pin their analyses to a specific, documented software release.

Some of the post-processing efforts were confounded by the way in which Coreform used interpolation of field results data to tetrahedral meshes for visualization in ParaView. This interpolation introduced artifacts that complicated the interpretation of results, particularly

for the nonlinear analyses. Post-processing maturity matters as much as solver performance when judging whether a platform is ready for production use. We found the experience of working with the post-processing workflow to be a source of friction that undermined our confidence in the numerical results we were seeing.

4.3. Distinguishing IGA as a Technology from Coreform as a Platform

This project taught us to distinguish IGA, a mathematical and computational technology, from Coreform IGA, a specific software implementation of it. Our experience with Coreform IGA—including the learning curve, the license access difficulties, the input deck compatibility issues, and the memory limitations encountered on the circuit board model—reflects the current maturity of one particular platform. It does not reflect fundamental limitations of IGA itself.

This distinction shapes how Sandia should read the successes and shortcomings in this report. The convergence advantages demonstrated for the C-Frame are a property of the IGA discretization; they would be expected to persist regardless of which IGA solver is used. The operational friction described above is largely a property of Coreform IGA’s relatively nascent development stage.

Coreform has recently shifted its primary development focus away from Coreform IGA as a standalone solver. As of early 2026, Coreform’s active development is directed toward an Abaqus plug-in, enabling IGA capabilities within the Dassault Systèmes SIMULIA ecosystem through the Coreform–SIMULIA partner program. This shift suggests Coreform IGA, as it stands today, may not be the long-term path to production-grade IGA analysis at Sandia.

4.4. Path Forward for IGA at Sandia

Coreform IGA had real operational problems. We remain optimistic about IGA’s long-term potential at Sandia, and we offer the following observations on the most promising paths forward.

The most pragmatic near-term path for IGA adoption at Sandia is likely incremental integration within the existing Sierra Solid Mechanics (SSM) and Sierra Solid Dynamics (SSD) framework. Sandia analysts already use SSM and SSD daily, and both solvers carry decades of validation. Extending these solvers to accept spline-based discretizations—building on the NURBS and Bézier extraction technologies that are well-established in the literature—would allow analysts to leverage IGA accuracy advantages without abandoning the tools and workflows they know. This path avoids the software-maturity risks associated with research-grade or early commercial platforms.

A second promising path is integration with the MOOSE framework through the INL–Coreform collaboration described in Section 2.2.1 of this report. MOOSE is open-source and modular, and its IGA development is active, including GPU-accelerated solvers under

development through DOE SBIR funding. That makes MOOSE well suited to multiphysics IGA applications. Were a robust integration between MOOSE's IGA capabilities and Sandia's Sierra ecosystem to be established, it could provide a high-performance, government-supported IGA solution that sidesteps the commercial software maturity concerns raised above.

In both cases, this project delivered more than the quantitative comparisons in this report. We also built real experience with IGA workflows, tooling, and problem setup. That experience positions Sandia to engage more effectively with IGA technology as it continues to mature across the industry, academia, and the national laboratory community.

DRAFT

This page intentionally left blank.

5. CONCLUSION

This report compared IGA to traditional FEA on three Sandia engineering components: the C-Frame, door hinge, and circuit board.

For the C-Frame modal analysis, IGA demonstrated clear advantages in both convergence rate and computational efficiency. The IGA discretization achieved comparable modal frequency accuracy at substantially lower DOF counts than either the SSD tet10 or SSD hex8 formulations, and required less total processor time to reach a given level of accuracy. These results match theoretical expectations for higher-order, higher-continuity spline bases [9, 10]. The door hinge and circuit board models extended the study to nonlinear static and modal analysis regimes; while a full multi-resolution convergence characterization was not performed for these geometries, the single-level comparisons conducted are consistent with the trends established by the C-Frame.

Coreform IGA works, but it has real friction: workflow setup, license access, input deck stability across versions, and post-processing fidelity. That friction comes from Coreform IGA's relative immaturity as a platform, not from any limitation in IGA as a technology. IGA's performance advantages are real and geometry-independent. Coreform IGA's operational problems are platform-specific and should improve as the software matures.

We recommend that Sandia pursue IGA through incremental integration in the existing SSM/SSD framework, which analysts already know and trust. Engagement with the MOOSE-IGA ecosystem, currently under active development through the INL-Coreform SBIR partnership, represents a complementary path worth monitoring.

IGA delivers on its core promise: better accuracy per degree of freedom. Sandia's engineering analysis community should benefit as adoption continues. The remaining barriers aren't mathematical. They're organizational: mature software toolchains, deeper institutional knowledge, and clear integration points with existing workflows. We hope this report is a first step toward clearing them.

This page intentionally left blank.

REFERENCES

- [1] Altair. *Altair HyperWorks 2026 Release Notes: Warp-map tools and IGA-like capabilities*. Altair Engineering, Inc., 2026. <https://altair.com/hyperworks-2026> (Accessed: Jan. 16, 2026)
- [2] M. J. Borden, M. A. Scott, J. A. Evans, and T. J. R. Hughes. Isogeometric finite element data structures based on Bézier extraction of NURBS. *International Journal for Numerical Methods in Engineering*, 87(1-5):15–47, 2011. <https://doi.org/10.1002/nme.2968>
- [3] Coreform LLC. *Phase II SBIR: Accelerating Nuclear Energy Engineering through GPU-enabled IGA within MOOSE*. U.S. Department of Energy, 2025.
- [4] Coreform LLC. *About Coreform: Leadership and Advisory Board*. <https://coreform.com/about/>. <https://coreform.com/about/> (Accessed: Jan. 16, 2026.)
- [5] L. Dalcin, N. Collier, P. Vignal, A. Cortes, and V. M. Calo. PetIGA: High-performance isogeometric analysis. *King Abdullah University of Science and Technology (KAUST) Extreme Computing Research Center*, 2016.
- [6] C. de Falco, A. Reali, and R. Vázquez. GeoPDEs: a package for Isogeometric Analysis in Octave and Matlab. *IMATI-CNR (Pavia, Italy)*, 2011.
- [7] G+Smo Wiki. *Austrian National Research Network “Geometry + Simulation,”* funded by the Austrian Science Fund (FWF). University of West Bohemia Developer Workshop, January 2026.
- [8] Hovey, C.B. (2022). *Formulation and Python Implementation of Bézier and B-Spline Geometry*. SAND2022-7702C. Sandia National Laboratories, June 8, 2022. <https://doi.org/10.5281/zenodo.6885108>. (Accessed: Jan. 16, 2026.)
- [9] T. J. R. Hughes, J. A. Cottrell, and Y. Bazilevs. Isogeometric analysis: CAD, finite elements, NURBS, exact geometry and mesh refinement. *Computer Methods in Applied Mechanics and Engineering*, 194(39-41):4135–4195, 2005. <https://doi.org/10.1016/j.cma.2004.10.008>
- [10] T. J. R. Hughes. *Isogeometric Analysis: Past, Present, and Future*. NCSU Distinguished Lecture Series, 2022. <https://youtu.be/ZtlgEUYGZMA> (Accessed: Mar. 27, 2026.)
- [11] Idaho National Laboratory. *Partnering with Coreform LLC for IGA capabilities in MOOSE*. U.S. Department of Energy SBIR Awards, 2021.

- [12] B. Jüttler, U. Zore, J. Barič, A. Mantzaflaris, B. Simeon, and R. Vázquez. G+Smo (Geometry + Simulation Modules): A comprehensive, open-source C++ library for IGA. *Johannes Kepler University Linz and RICAM Institute*, 2014.
- [13] R. C. Martineau et al. *Multiphysics Object-Oriented Simulation Environment (MOOSE)*. Idaho National Laboratory (INL), 2021.
- [14] MOOSE Documentation. *IGA applied to a C-frame structural analysis problem*. Multiphysics Object-Oriented Simulation Environment, (Accessed: Mar. 27, 2026).
- [15] B. Novozhilov and P. Leidinger. *Trimmed IGA Solids: Performance on Thin-Walled Structures with Complex Geometry*. 2025 Ansys EMEA Transportation Summit and LS-DYNA Conference, Munich, Germany, 2025. Ansys, Inc., 2025.
- [16] M. A. Scott, T. Blacker, and T. J. R. Hughes. *The Design-to-Analysis Workflow: A Motivation for Isogeometric Analysis*. Sandia National Laboratories, 2011.
- [17] Siemens Digital Industries Software. *NX Performance Predictor and Subdivision Modeling Updates. What’s New In NX June 2025*. <https://plm.sw.siemens.com/en-US/nx/cad-online/latest-nx-cad-version/>. (Accessed: Jan. 16, 2026). Siemens, 2025.
- [18] R. Vázquez. A new design for GeoPDEs, a MATLAB/Octave toolbox for isogeometric analysis. *University of Santiago de Compostela*, 2016.

APPENDIX A. Basis Functions

The representation of geometry and solution fields in numerical analysis relies on the choice of basis functions. In traditional Finite Element Analysis (FEA), the standard approach uses **Lagrange Polynomials**. These “nodal” basis functions are defined by interpolation at specific points (nodes) within an element. They satisfy the Kronecker delta property:

$$L_i(x_j) = \delta_{ij} = \begin{cases} 1 & \text{if } i = j \\ 0 & \text{if } i \neq j \end{cases} \quad (\text{A.1})$$

This property ensures that the coefficient of a basis function corresponds *directly to the physical value* at that node.

A.1. Lagrange Basis Functions

Lagrange interpolations are effective for low-order elements (h -refinement). However, for higher-order elements, with increasing polynomial order (p -refinement) and equally spaced nodes, Lagrange interpolations lead to numerical instability. High-order Lagrange interpolants exhibit **Runge’s Phenomenon**, where the polynomial oscillates wildly near the edges of the interval. This results in poor approximation properties and ill-conditioned systems of equations. Higher-order Lagrange elements are often not used in practice, and instead, low-order elements with mesh refinement (“ h -type”) are preferred.

The general formula for a Lagrange polynomial of degree n is:

$$L_i(x) = \prod_{j=0, j \neq i}^n \frac{x - x_j}{x_i - x_j} \quad (\text{A.2})$$

For a standard reference element $x \in [-1, 1]$, the explicit basis functions are:

Linear Elements (2 nodes at $x = \{-1, 1\}$):

$$L_0(x) = \frac{1 - x}{2} \quad (\text{A.3})$$

$$L_1(x) = \frac{1 + x}{2} \quad (\text{A.4})$$

Quadratic Elements (3 nodes at $x = \{-1, 0, 1\}$):

$$L_0(x) = \frac{x^2 - x}{2} \quad (\text{A.5})$$

$$L_1(x) = 1 - x^2 \quad (\text{A.6})$$

$$L_2(x) = \frac{x^2 + x}{2} \quad (\text{A.7})$$

Cubic Elements (4 nodes at $x = \{-1, -\frac{1}{3}, \frac{1}{3}, 1\}$):

$$L_0(x) = -\frac{9}{16} \left(x^3 - x^2 - \frac{1}{9}x + \frac{1}{9} \right) \quad (\text{A.8})$$

$$L_1(x) = \frac{27}{16} \left(x^3 - \frac{1}{3}x^2 - x + \frac{1}{3} \right) \quad (\text{A.9})$$

$$L_2(x) = -\frac{27}{16} \left(x^3 + \frac{1}{3}x^2 - x - \frac{1}{3} \right) \quad (\text{A.10})$$

$$L_3(x) = \frac{9}{16} \left(x^3 + x^2 - \frac{1}{9}x - \frac{1}{9} \right) \quad (\text{A.11})$$

Figure A-1 illustrates these functions.

A.2. Legendre (GLL) Basis Functions

To mitigate the instability of high-order Lagrange elements, **Legendre Interpolation** (specifically using Gauss-Lobatto-Legendre or GLL nodes) is employed. Instead of equal spacing, nodes are clustered toward the endpoints, coinciding with the roots of $(1 - x^2)P'_n(x)$, where $P_n(x)$ is the Legendre polynomial. This clustering significantly reduces Runge's Phenomenon, providing a stable basis for high-order spectral element methods.

The specific GLL polynomials presented here are still interpolatory (nodal) and satisfy the Kronecker delta property (Eq. A.1). They maintain numerical stability over the $[-1, 1]$ interval, which allows for efficient integration and better convergence in p -refinement strategies.

Linear GLL (nodes at $x = \{-1, 1\}$): Identical to Linear Lagrange.

Quadratic GLL (nodes at $x = \{-1, 0, 1\}$): Identical to Quadratic Lagrange.

Cubic GLL (nodes at $x = \{-1, -\sqrt{1/5}, \sqrt{1/5}, 1\}$): Given the nodes $\xi_1 = -\sqrt{0.2}$ and

$\xi_2 = \sqrt{0.2}$, the basis functions are:

$$L_0(x) = \frac{(x - \xi_1)(x - \xi_2)(x - 1)}{(-1 - \xi_1)(-1 - \xi_2)(-1 - 1)} \quad (\text{A.12})$$

$$L_1(x) = \frac{(x + 1)(x - \xi_2)(x - 1)}{(\xi_1 + 1)(\xi_1 - \xi_2)(\xi_1 - 1)} \quad (\text{A.13})$$

$$L_2(x) = \frac{(x + 1)(x - \xi_1)(x - 1)}{(\xi_2 + 1)(\xi_2 - \xi_1)(\xi_2 - 1)} \quad (\text{A.14})$$

$$L_3(x) = \frac{(x + 1)(x - \xi_1)(x - \xi_2)}{(1 + 1)(1 - \xi_1)(1 - \xi_2)} \quad (\text{A.15})$$

Note: For the Linear and Quadratic cases, $n = 1$ and $n = 2$, GLL nodes coincide with equally spaced nodes. Therefore, the Linear and Quadratic Legendre bases are mathematically identical to the Linear and Quadratic Lagrange bases, respectively.

Figure A-2 illustrates these functions.

A.3. Bézier (Bernstein) Basis Functions

In Isogeometric Analysis (IGA), **Bernstein Polynomials** serve as the basis for Bézier curves, surfaces, and volumes. Unlike Lagrange and Legendre bases, Bernstein polynomials are **not interpolatory** (except for at the first and last control points in a single patch). The resulting curve does not pass through the interior control points. Bernstein polynomials provide a smooth approximation that remains strictly within the convex hull of the control points. See Hovey [8] for additional details.

Key properties include **Positivity** (the functions are always ≥ 0) and **Partition of Unity** (they sum to 1 at any point). These properties prevent the oscillations seen in Lagrange interpolation, ensuring geometric stability.

The Bernstein basis functions of degree n are defined on $t \in [0, 1]$ as:

$$B_{i,n}(t) = \binom{n}{i} t^i (1 - t)^{n-i} \quad (\text{A.16})$$

To map these to the reference element $x \in [-1, 1]$, we use the transformation $t = \frac{x+1}{2}$.

Linear Bézier ($n = 1$): Identical to Linear Lagrange and Linear Legendre.

$$B_{0,1}(x) = \frac{1 - x}{2} \quad (\text{A.17})$$

$$B_{1,1}(x) = \frac{1 + x}{2} \quad (\text{A.18})$$

Quadratic Bézier ($n = 2$):

$$B_{0,2}(x) = \frac{(1-x)^2}{4} \quad (\text{A.19})$$

$$B_{1,2}(x) = \frac{1-x^2}{2} \quad (\text{A.20})$$

$$B_{2,2}(x) = \frac{(1+x)^2}{4} \quad (\text{A.21})$$

Cubic Bézier ($n = 3$):

$$B_{0,3}(x) = \frac{(1-x)^3}{8} \quad (\text{A.22})$$

$$B_{1,3}(x) = \frac{3(1-x)^2(1+x)}{8} \quad (\text{A.23})$$

$$B_{2,3}(x) = \frac{3(1-x)(1+x)^2}{8} \quad (\text{A.24})$$

$$B_{3,3}(x) = \frac{(1+x)^3}{8} \quad (\text{A.25})$$

Figure A-3 illustrates these functions.

The B-spline linear functions match the linear functions of traditional FEA. However, starting with the quadratic basis and above, B-spline functions no longer appear like they do in finite elements.

The quadratic and higher B-spline functions have larger support (wider knot spans). This gives better accuracy per degree of freedom, especially for problems with higher-order derivatives or smooth solution fields [10].

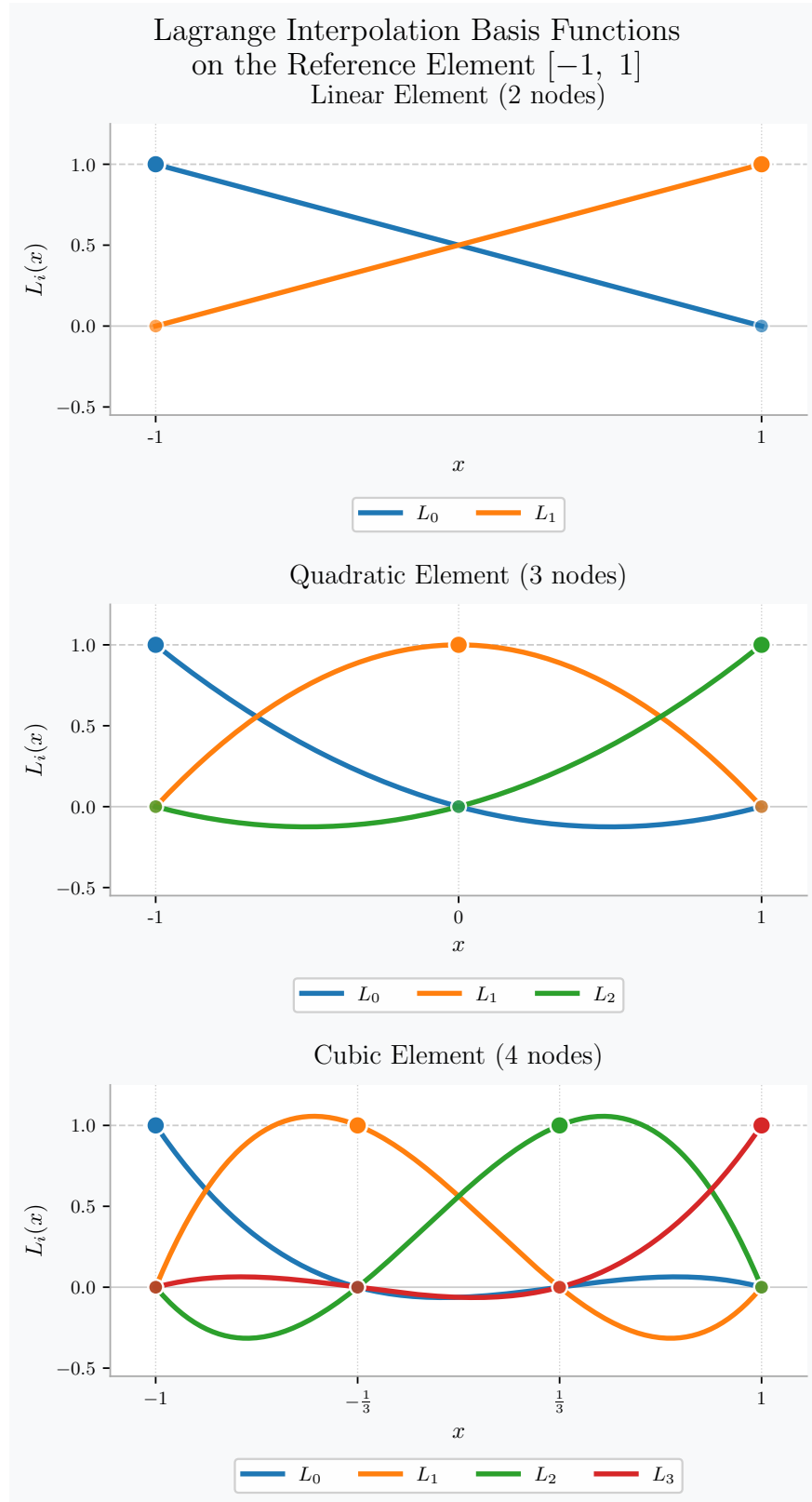


Figure A-1. Lagrange basis functions for 2-, 3-, and 4-noded 1D element.

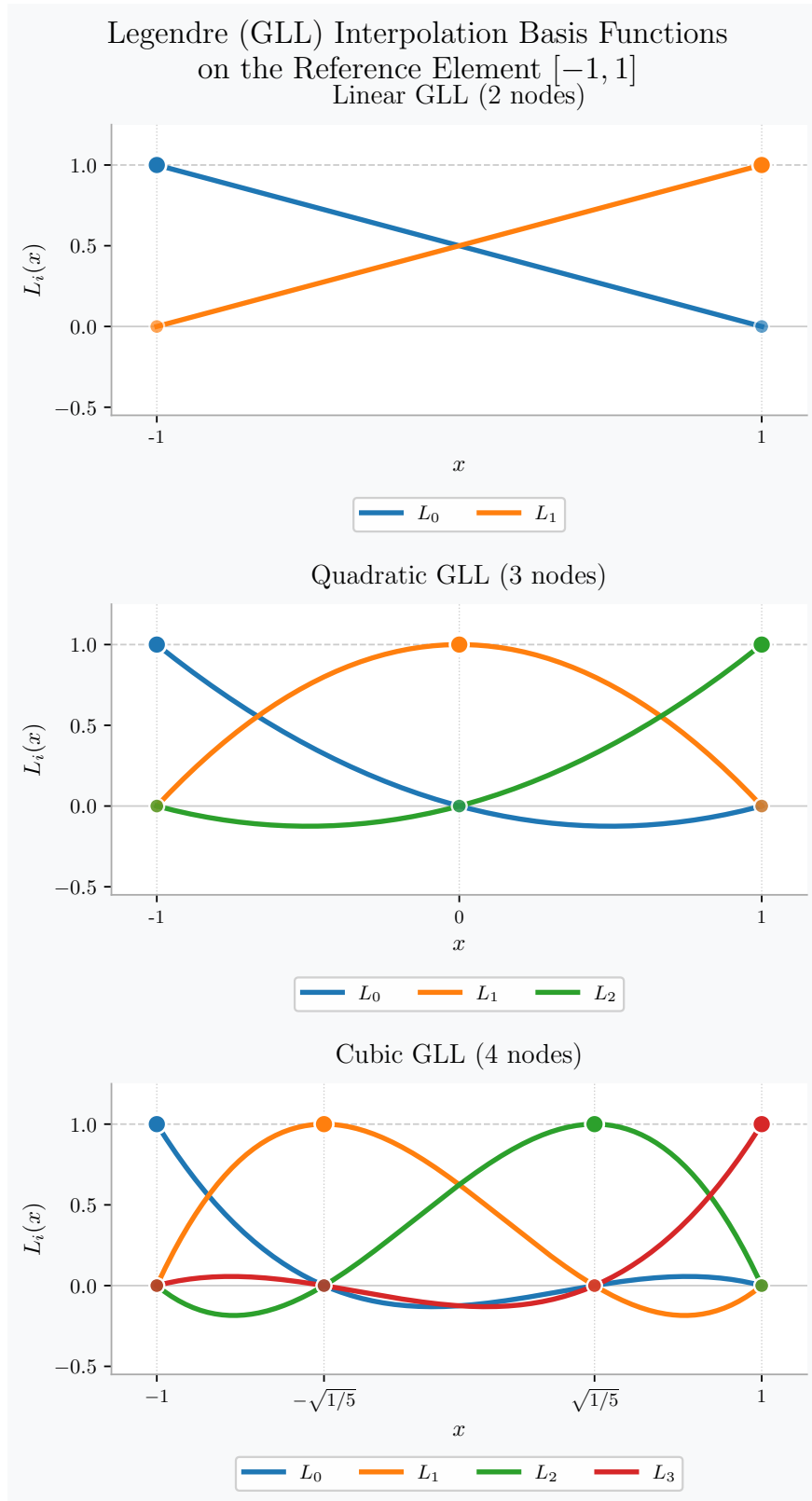


Figure A-2. Legendre basis functions for 2-, 3-, and 4-noded 1D element.

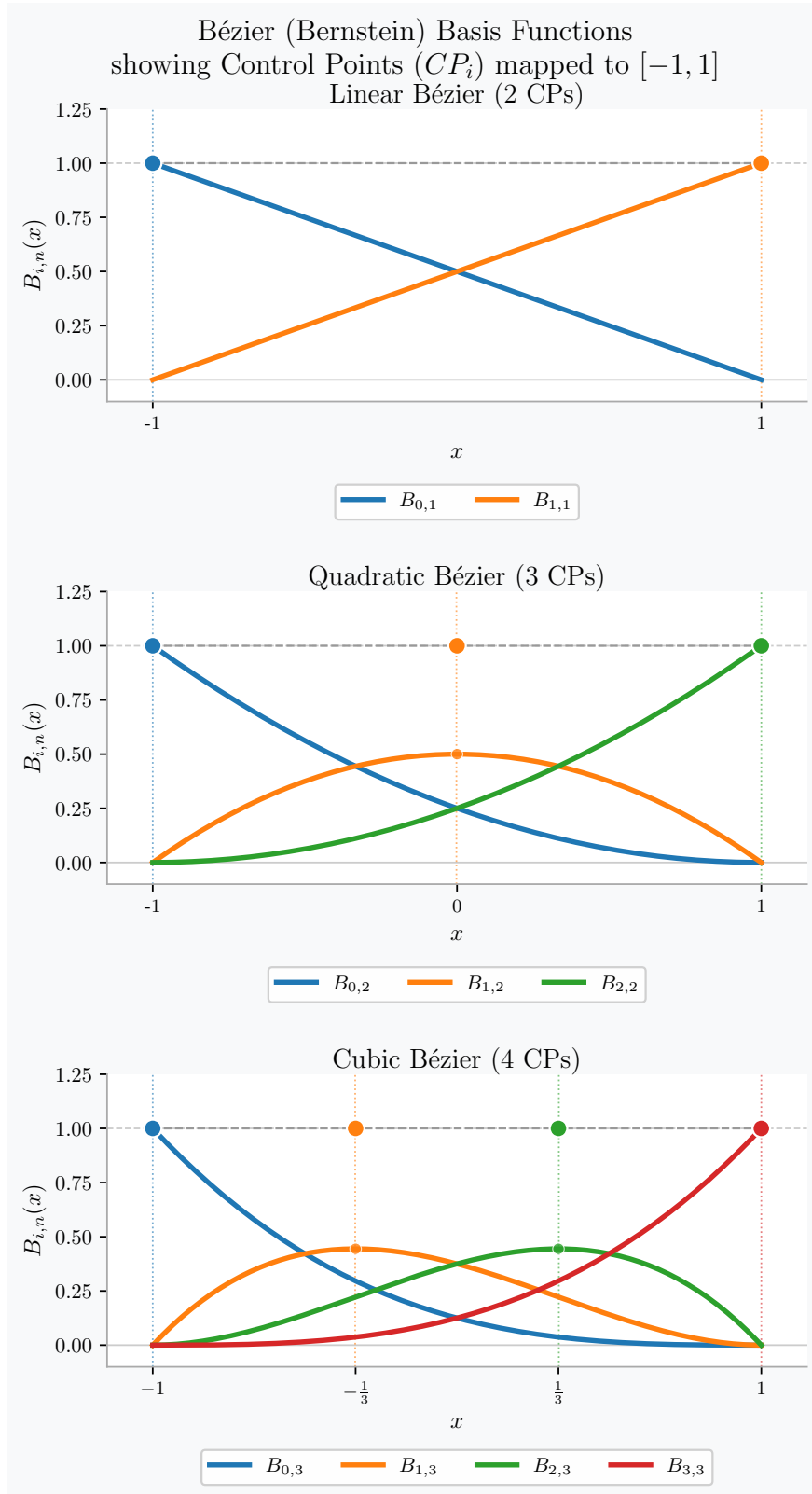


Figure A-3. Bézier basis functions for 2-, 3-, and 4-noded 1D element.

This page intentionally left blank.

APPENDIX B. Bézier Extraction

Bézier extraction is a technique that lets standard (traditional) finite element solvers, typically built to handle Lagrange polynomial basis functions, solve problems discretized with spline-based basis functions such as NURBS and T-splines [10]. Bézier extraction is a mathematical operator that converts globally defined B-spline or NURBS basis functions into a set of locally supported Bernstein polynomials, which define Bézier elements. This lets standard finite element assembly and solution techniques work with spline-based discretizations, without modification to the underlying solver.

Borden et al. [2] present the Bézier extraction operator and isogeometric Bézier elements for NURBS-based isogeometric analysis. The Bézier extraction operator lets numerical integration of smooth functions be performed on C^0 Bézier elements. As they put it, Bézier elements “provide an element structure for isogeometric analysis that can be easily incorporated into existing finite element codes, without any changes to element form and assembly algorithms, and standard data processing arrays” [2].

Borden et al. also describe the extraction operator directly: “the Bézier extraction operator maps a piecewise Bernstein polynomial basis onto a B-spline basis. This transformation makes it possible to use piecewise C^0 Bézier elements as the finite element representation of an NURBS or T-spline” [2].

This page intentionally left blank.

APPENDIX C. IGA Input Files

C.1. C-Frame

C.1.1. jobs_coarse.iga

```
1 {
2   "coreform_iga_version": "2025.7",
3   "data_output_file": "cf_iga_data_output.json",
4   "data_output_interval": "intervals_1",
5   "flex_models": [
6     {
7       "database_name": "geom",
8       "label": "flex_models_1",
9       "parts": [
10        {
11          "label": "parts_1",
12          "material": "materials_1",
13          "material_model": "elastic",
14          "part": "part_1",
15          "quadrature_options": {
16            "optimization_lawson_hanson_solve_completion": true,
17            "optimization_method": "tet_pruning",
18            "optimization_residual_tolerance": 1.0000000000000002e-12,
19            "optimized_cut_quadrature_strategy": "reduce_from_uncut",
20            "quadrature": "QP1"
21          }
22        }
23      ],
24      "small_cell_volume_ratio": 0
25    },
26  ],
27  "functions": [
28    {
29      "constant": {
30        "value": 1
31      },
32      "label": "functions_1"
33    }
34  ],
35  "intervals": [
36    {
37      "label": "intervals_1",
38      "start_step": 0,
39      "step_increment": 1,
40      "stop_step": 20,
41      "time_increment": 1
42    }
43  ],
44  "label": "iga_1",
45  "materials": [
46    {
47      "elastic": {
48        "poissons_ratio": 0.29,
49        "youngs_modulus": 28000000
50      },
51      "label": "materials_1",
52      "mass_density": 0.000735
53    }
54  ],
55  "procedures": [
56    {
57      "label": "procedures_1",
58      "structural_modal_analysis": {
59        "boundary_conditions": [
60          "boundary_conditions_1"
61        ],
62        "flex_model": "flex_models_1",
63        "modal_solution_method": "modal_solution_methods_1",
64        "outputs": [
65          "outputs_1"
66        ]
67      }
68    }
69  ]
70 }
```

```

67     }
68   },
69 ],
70 "structural_modal_analysis_definitions": {
71   "boundary_conditions": [
72     {
73       "components": [
74         "x",
75         "y",
76         "z"
77       ],
78       "label": "boundary_conditions_1",
79       "penalty": {
80         "dimensionless": true,
81         "value": 10
82       },
83       "quadrature_options": {
84         "optimization_lawson_hanson_solve_completion": true,
85         "optimization_method": "tet_pruning",
86         "optimization_residual_tolerance": 1.0000000000000002e-12,
87         "optimized_cut_quadrature_strategy": "reduce_from_uncut",
88         "quadrature": "QP1"
89       },
90       "set": "fixed"
91     }
92   ],
93   "modal_solution_methods": [
94     {
95       "frequency_extraction": {
96         "maximum_frequency": 4000,
97         "minimum_frequency": 1
98       },
99       "label": "modal_solution_methods_1"
100     }
101   ],
102   "outputs": [
103     {
104       "field": {
105         "database_name": "cframe_modal_coarse",
106         "external_surfaces_only": false,
107         "include_integration_point_output": false,
108         "interval": "intervals_1",
109         "part": "part_1",
110         "variables": {
111           "displacement": [
112             "all"
113           ]
114         }
115       },
116       "label": "outputs_1",
117       "tessellate": true
118     }
119   ]
120 }
121 }

```

C.1.2. jobs_fine.iga

```

1 {
2   "coreform_iga_version": "2025.7",
3   "data_output_file": "cf_iga_data_output.json",
4   "data_output_interval": "intervals_1",
5   "flex_models": [
6     {
7       "database_name": "geom",
8       "label": "flex_models_1",
9       "parts": [
10        {
11          "label": "parts_1",
12          "material": "materials_1",
13          "material_model": "elastic",
14          "part": "part_1",
15          "quadrature_options": {
16            "optimization_lawson_hanson_solve_completion": true,
17            "optimization_method": "tet_pruning",
18            "optimization_residual_tolerance": 1.0000000000000002e-12,
19            "optimized_cut_quadrature_strategy": "reduce_from_uncut",
20            "quadrature": "QP1"
21          }
22        }

```

```

23     ],
24     "small_cell_volume_ratio": 0
25   }
26 ],
27 "functions": [
28   {
29     "constant": {
30       "value": 1
31     },
32     "label": "functions_1"
33   }
34 ],
35 "intervals": [
36   {
37     "label": "intervals_1",
38     "start_step": 0,
39     "step_increment": 1,
40     "stop_step": 20,
41     "time_increment": 1
42   }
43 ],
44 "label": "iga_1",
45 "materials": [
46   {
47     "elastic": {
48       "poissons_ratio": 0.29,
49       "youngs_modulus": 28000000
50     },
51     "label": "materials_1",
52     "mass_density": 0.000735
53   }
54 ],
55 "procedures": [
56   {
57     "label": "procedures_1",
58     "structural_modal_analysis": {
59       "boundary_conditions": [
60         "boundary_conditions_1"
61       ],
62       "flex_model": "flex_models_1",
63       "modal_solution_method": "modal_solution_methods_1",
64       "outputs": [
65         "outputs_1"
66       ]
67     }
68   }
69 ],
70 "structural_modal_analysis_definitions": {
71   "boundary_conditions": [
72     {
73       "components": [
74         "x",
75         "y",
76         "z"
77       ],
78       "label": "boundary_conditions_1",
79       "penalty": {
80         "dimensionless": true,
81         "value": 10
82       },
83       "quadrature_options": {
84         "optimization_lawson_hanson_solve_completion": true,
85         "optimization_method": "tet_pruning",
86         "optimization_residual_tolerance": 1.0000000000000002e-12,
87         "optimized_cut_quadrature_strategy": "reduce_from_uncut",
88         "quadrature": "QP1"
89       },
90       "set": "fixed"
91     }
92   ],
93   "modal_solution_methods": [
94     {
95       "frequency_extraction": {
96         "maximum_frequency": 4000,
97         "minimum_frequency": 1
98       },
99       "label": "modal_solution_methods_1"
100     }
101   ],
102   "outputs": [
103     {
104       "field": {
105         "database_name": "cframe_modal_fine",
106         "external_surfaces_only": false,
107         "include_integration_point_output": false,
108         "interval": "intervals_1",
109         "part": "part_1",
110         "variables": {
111           "displacement": [
112             "all"
113           ]
114         }
115       }
116     }
117   ]
118 }

```

```

114     }
115   },
116   "label": "outputs_1",
117   "tessellate": true
118 }
119 ]
120 }
121 }

```

C.1.3. jobs_fine_x2.iga

```

1 {
2   "coreform_iga_version": "2025.7",
3   "data_output_file": "cf_iga_data_output.json",
4   "data_output_interval": "intervals_1",
5   "flex_models": [
6     {
7       "database_name": "geom",
8       "label": "flex_models_1",
9       "parts": [
10        {
11          "label": "parts_1",
12          "material": "materials_1",
13          "material_model": "elastic",
14          "part": "part_1",
15          "quadrature_options": {
16            "optimization_lawson_hanson_solve_completion": true,
17            "optimization_method": "tet_pruning",
18            "optimization_residual_tolerance": 1.0000000000000002e-12,
19            "optimized_cut_quadrature_strategy": "reduce_from_uncut",
20            "quadrature": "QP1"
21          }
22        }
23      ],
24      "small_cell_volume_ratio": 0
25    }
26  ],
27  "functions": [
28    {
29      "constant": {
30        "value": 1
31      },
32      "label": "functions_1"
33    }
34  ],
35  "intervals": [
36    {
37      "label": "intervals_1",
38      "start_step": 0,
39      "step_increment": 1,
40      "stop_step": 20,
41      "time_increment": 1
42    }
43  ],
44  "label": "iga_1",
45  "materials": [
46    {
47      "elastic": {
48        "poissons_ratio": 0.29,
49        "youngs_modulus": 28000000
50      },
51      "label": "materials_1",
52      "mass_density": 0.000735
53    }
54  ],
55  "procedures": [
56    {
57      "label": "procedures_1",
58      "structural_modal_analysis": {
59        "boundary_conditions": [
60          "boundary_conditions_1"
61        ],
62        "flex_model": "flex_models_1",
63        "modal_solution_method": "modal_solution_methods_1",
64        "outputs": [
65          "outputs_1"
66        ]
67      }
68    }
69  ],

```

```

70 "structural_modal_analysis_definitions": {
71   "boundary_conditions": [
72     {
73       "components": [
74         "x",
75         "y",
76         "z"
77       ],
78       "label": "boundary_conditions_1",
79       "penalty": {
80         "dimensionless": true,
81         "value": 10
82       },
83       "quadrature_options": {
84         "optimization_lawson_hanson_solve_completion": true,
85         "optimization_method": "tet_pruning",
86         "optimization_residual_tolerance": 1.0000000000000002e-12,
87         "optimized_cut_quadrature_strategy": "reduce_from_uncut",
88         "quadrature": "QP1"
89       },
90       "set": "fixed"
91     }
92   ],
93   "modal_solution_methods": [
94     {
95       "frequency_extraction": {
96         "maximum_frequency": 4000,
97         "minimum_frequency": 1
98       },
99       "label": "modal_solution_methods_1"
100     }
101   ],
102   "outputs": [
103     {
104       "field": {
105         "database_name": "cframe_modal_fine_x2",
106         "external_surfaces_only": false,
107         "include_integration_point_output": false,
108         "interval": "intervals_1",
109         "part": "part_1",
110         "variables": {
111           "displacement": [
112             "all"
113           ]
114         }
115       },
116       "label": "outputs_1",
117       "tessellate": true
118     }
119   ]
120 }
121 }

```

C.2. Door Hinge

C.2.1. door_hinge_failure_workflow.txt

```

1  #=Door Lock Hinge Assembly Problem
2  reset
3  dev rimport "/tutorials/door_lock_assembly/door_lock_assembly.sat"
4  #!=Door Lock Hinge Assembly Problem
5  #=DEFINE PARTS AND SETS
6  name part 1 "door_link"
7  name part 2 "door_hinge"
8  name part 3 "door_pin"
9  set "door_link_bottom_surfaces" add surfaces [21 29]
10 set "door_hinge_bolt_surfaces" add surfaces [111 115]
11 #!=DEFINE PARTS AND SETS
12 #=MESH GENERATION
13 mesh "mesh_door_link" rectilinear degree 2 continuity 1 element_size [1 1 1] padding [2 2 2]
14 part "door_link" mesh "mesh_door_link"
15 part "door_link" volume_box
16 mesh "mesh_door_hinge" rectilinear degree 2 continuity 1 element_size [1 1 1] padding [2 2 2]
17 part "door_hinge" mesh "mesh_door_hinge"
18 part "door_hinge" volume_box
19 mesh "mesh_door_pin" rectilinear degree 2 continuity 1 element_size [1 1 1] padding [2 2 2]
20 part "door_pin" mesh "mesh_door_pin"

```

```

21 part "door_pin" volume_box
22 #!=MESH GENERATION
23 #=FUNCTIONS
24 functions "aluminum_plastic_hardening" new
25 functions "aluminum_plastic_hardening" piecewise_linear abscissa [0 10]
26 functions "aluminum_plastic_hardening" piecewise_linear ordinate [0 100]
27 functions "steel_plastic_hardening" new
28 functions "steel_plastic_hardening" piecewise_linear abscissa [0 10]
29 functions "steel_plastic_hardening" piecewise_linear ordinate [0 100]
30 functions "constant_1" new
31 functions "constant_1" constant value 1
32 functions "linear_ramp" new
33 functions "linear_ramp" piecewise_linear abscissa [0 1]
34 functions "linear_ramp" piecewise_linear ordinate [0 1]
35 #!=FUNCTIONS
36 #=MATERIALS
37 materials "aluminum" new
38 materials "aluminum" mass_density 2.7e-9
39 materials "aluminum" neohookean_isotropic_plasticity neohookean youngs_modulus 73.8e3
40 materials "aluminum" neohookean_isotropic_plasticity neohookean poissons_ratio 0.3
41 materials "aluminum" neohookean_isotropic_plasticity isotropic_plasticity yield_stress 322
42 materials "aluminum" neohookean_isotropic_plasticity isotropic_plasticity hardening_function "aluminum_plastic_hardening"
43 materials "steel" new
44 materials "steel" mass_density 2.7e-9
45 materials "steel" neohookean_isotropic_plasticity neohookean youngs_modulus 200e3
46 materials "steel" neohookean_isotropic_plasticity neohookean poissons_ratio 0.29
47 materials "steel" neohookean_isotropic_plasticity isotropic_plasticity yield_stress 2e3
48 materials "steel" neohookean_isotropic_plasticity isotropic_plasticity hardening_function "steel_plastic_hardening"
49 tessellation_options "tessellation_options_1" meshgems average_triangle_edge_subdivisions 4
50 #!=MATERIALS
51 #=FLEX MODELS
52 flex_models "flex_models_1" new
53 flex_models "flex_models_1" database_name "geom"
54 flex_models "flex_models_1" small_cell_volume_ratio 0
55 flex_models "flex_models_1" parts "door_hinge_instance" part "door_hinge"
56 flex_models "flex_models_1" parts "door_hinge_instance" material "aluminum"
57 flex_models "flex_models_1" parts "door_hinge_instance" material_model neohookean_isotropic_plasticity
58 flex_models "flex_models_1" parts "door_hinge_instance" quadrature_options tessellation_option "tessellation_options_1"
59 flex_models "flex_models_1" parts "door_link_instance" part "door_link"
60 flex_models "flex_models_1" parts "door_link_instance" material "aluminum"
61 flex_models "flex_models_1" parts "door_link_instance" material_model neohookean_isotropic_plasticity
62 flex_models "flex_models_1" parts "door_link_instance" quadrature_options tessellation_option "tessellation_options_1"
63 flex_models "flex_models_1" parts "door_pin_instance" part "door_pin"
64 flex_models "flex_models_1" parts "door_pin_instance" material "steel"
65 flex_models "flex_models_1" parts "door_pin_instance" material_model neohookean_isotropic_plasticity
66 flex_models "flex_models_1" parts "door_pin_instance" quadrature_options tessellation_option "tessellation_options_1"
67 #!=FLEX MODELS
68 #=INTERVALS
69 intervals "simulation_interval" new
70 intervals "simulation_interval" start_time 0
71 intervals "simulation_interval" stop_time 1
72 intervals "simulation_interval" time_increment 5e-4
73 intervals "simulation_output_interval" new
74 intervals "simulation_output_interval" use_start_stop_from_interval "simulation_interval"
75 intervals "simulation_output_interval" step_increment 1
76 #!=INTERVALS
77 #=LINEAR EQUATION SOLVERS
78 linear_equation_solvers "direct_lu" new
79 linear_equation_solvers "direct_lu" direct lu
80 linear_equation_solvers "direct_multi_frontal" new
81 linear_equation_solvers "direct_multi_frontal" direct multi_frontal
82 #!=LINEAR EQUATION SOLVERS
83 #=NONLINEAR EQUATION SOLVERS
84 nonlinear_equation_solvers "newton_raphson" new
85 nonlinear_equation_solvers "newton_raphson" newton linear_equation_solver "direct_multi_frontal"
86 nonlinear_equation_solvers "newton_raphson" newton maximum_iterations 20
87 nonlinear_equation_solvers "newton_raphson" newton target_relative_residual 1e-8
88 #!=NONLINEAR EQUATION SOLVERS
89 #=TIMESTEPPERS
90 time_steppers "implicit_dynamics_quasistatic" new
91 time_steppers "implicit_dynamics_quasistatic" generalized_alpha spectral_radius 0
92 time_steppers "implicit_dynamics_quasistatic" generalized_alpha predictor constant_velocity
93 time_steppers "implicit_dynamics_quasistatic" generalized_alpha nonlinear_equation_solver "newton_raphson"
94 time_steppers "implicit_dynamics_quasistatic" generalized_alpha adaptivity maximum_time_step 0.1
95 time_steppers "implicit_dynamics_quasistatic" generalized_alpha adaptivity minimum_time_step 1e-5
96 time_steppers "implicit_dynamics_quasistatic" generalized_alpha adaptivity decrease_factor 0.25
97 time_steppers "implicit_dynamics_quasistatic" generalized_alpha adaptivity increase_factor 1.5
98 #!=TIMESTEPPERS
99 #=BOUNDARY CONDITIONS
100 solid_mechanics_definitions boundary_conditions "hold_door_link_bottom" new
101 solid_mechanics_definitions boundary_conditions "hold_door_link_bottom" set "door_link_bottom_surfaces"
102 solid_mechanics_definitions boundary_conditions "hold_door_link_bottom" displacement function "constant_1"
103 solid_mechanics_definitions boundary_conditions "hold_door_link_bottom" displacement components 0 x
104 solid_mechanics_definitions boundary_conditions "hold_door_link_bottom" displacement components 1 y
105 solid_mechanics_definitions boundary_conditions "hold_door_link_bottom" displacement components 2 z
106 solid_mechanics_definitions boundary_conditions "hold_door_link_bottom" displacement scale_factor 0
107 solid_mechanics_definitions boundary_conditions "hold_door_link_bottom" quadrature_options tessellation_option "tessellation_options_1"
108 solid_mechanics_definitions boundary_conditions "pull_door_hinge_bolts" new
109 solid_mechanics_definitions boundary_conditions "pull_door_hinge_bolts" set "door_hinge_bolt_surfaces"
110 solid_mechanics_definitions boundary_conditions "pull_door_hinge_bolts" displacement function "linear_ramp"
111 solid_mechanics_definitions boundary_conditions "pull_door_hinge_bolts" displacement components 0 x

```

```

112 solid_mechanics_definitions boundary_conditions "pull_door_hinge_bolts" displacement components 1 y
113 solid_mechanics_definitions boundary_conditions "pull_door_hinge_bolts" displacement components 2 z
114 solid_mechanics_definitions boundary_conditions "pull_door_hinge_bolts" displacement scale_factor [0 0 9]
115 solid_mechanics_definitions boundary_conditions "pull_door_hinge_bolts" quadrature_options tessellation_option "tessellation_options_1"
116 #!=BOUNDARY CONDITIONS
117 #!=INTERACTIONS
118 solid_mechanics_definitions interaction_properties "friction" new
119 solid_mechanics_definitions interaction_properties "friction" coulomb_friction coefficient_with_regularization friction_coefficient 0.3
120 solid_mechanics_definitions interaction_properties "friction" coulomb_friction coefficient_with_regularization regularization_velocity 0.1
121 solid_mechanics_definitions interactions "frictional_contact" new
122 solid_mechanics_definitions interactions "frictional_contact" mechanical_contact base_contact complement true
123 solid_mechanics_definitions interactions "frictional_contact" mechanical_contact interaction_properties "friction"
124 solid_mechanics_definitions interactions "frictional_contact" mechanical_contact approximate_tangent_iterations 6
125 solid_mechanics_definitions interactions "frictional_contact" quadrature_options tessellation_option "tessellation_options_1"
126 solid_mechanics_definitions interactions "frictional_contact" mechanical_contact activation_distance value 0.5
127 solid_mechanics_definitions interactions "frictional_contact" mechanical_contact activation_zone_influential_radius_factor 1.0
128 solid_mechanics_definitions interactions "frictional_contact" mechanical_contact potential_cutoff_distance_factor 0.5
129 #!=INTERACTIONS
130 #!=PROBES
131 solid_mechanics_definitions probes "door_link_reaction_force" new
132 solid_mechanics_definitions probes "door_link_reaction_force" integrated_surface_quantity
133 solid_mechanics_definitions probes "door_link_reaction_force" integrated_surface_quantity use_set_from_boundary_condition "hold_door_link_bottom"
134 solid_mechanics_definitions probes "door_link_reaction_force" integrated_surface_quantity variables reaction_force 0 x
135 solid_mechanics_definitions probes "door_link_reaction_force" integrated_surface_quantity variables reaction_force 1 y
136 solid_mechanics_definitions probes "door_link_reaction_force" integrated_surface_quantity variables reaction_force 2 z
137 solid_mechanics_definitions probes "door_hinge_reaction_force" new
138 solid_mechanics_definitions probes "door_hinge_reaction_force" integrated_surface_quantity
139 solid_mechanics_definitions probes "door_hinge_reaction_force" integrated_surface_quantity use_set_from_boundary_condition
140 ↪ "pull_door_hinge_bolts"
141 solid_mechanics_definitions probes "door_hinge_reaction_force" integrated_surface_quantity variables reaction_force 0 x
142 solid_mechanics_definitions probes "door_hinge_reaction_force" integrated_surface_quantity variables reaction_force 1 y
143 solid_mechanics_definitions probes "door_hinge_max_vm" new
144 solid_mechanics_definitions probes door_hinge_max_vm field extremum part active
145 solid_mechanics_definitions probes door_hinge_max_vm field extremum part ""
146 solid_mechanics_definitions probes door_hinge_max_vm field extremum part ""
147 solid_mechanics_definitions probes door_hinge_max_vm field extremum part "door_hinge"
148 solid_mechanics_definitions probes door_hinge_max_vm field extremum maximum active
149 solid_mechanics_definitions probes door_hinge_max_vm field extremum maximum stress active
150 solid_mechanics_definitions probes door_hinge_max_vm field extremum maximum stress von_mises
151 solid_mechanics_definitions probes door_hinge_max_vm field variables stress new
152 solid_mechanics_definitions probes door_hinge_max_vm field variables stress 0 von_mises
153 #!=PROBES
154 #!=FIELD OUTPUT
155 solid_mechanics_definitions outputs "door_link_field_results" new
156 solid_mechanics_definitions outputs "door_link_field_results" field database_name "door_link_results"
157 solid_mechanics_definitions outputs "door_link_field_results" field interval "simulation_output_interval"
158 solid_mechanics_definitions outputs "door_link_field_results" field part "door_link"
159 solid_mechanics_definitions outputs "door_link_field_results" field variables displacement 0 x
160 solid_mechanics_definitions outputs "door_link_field_results" field variables displacement 1 y
161 solid_mechanics_definitions outputs "door_link_field_results" field variables displacement 2 z
162 solid_mechanics_definitions outputs "door_link_field_results" field variables stress 0 von_mises
163 solid_mechanics_definitions outputs "door_link_field_results" field variables plastic_strain 0 equivalent
164 solid_mechanics_definitions outputs "door_hinge_field_results" new
165 solid_mechanics_definitions outputs "door_hinge_field_results" field database_name "door_hinge_results"
166 solid_mechanics_definitions outputs "door_hinge_field_results" field interval "simulation_output_interval"
167 solid_mechanics_definitions outputs "door_hinge_field_results" field part "door_hinge"
168 solid_mechanics_definitions outputs "door_hinge_field_results" field variables displacement 0 x
169 solid_mechanics_definitions outputs "door_hinge_field_results" field variables displacement 1 y
170 solid_mechanics_definitions outputs "door_hinge_field_results" field variables displacement 2 z
171 solid_mechanics_definitions outputs "door_hinge_field_results" field variables stress 0 von_mises
172 solid_mechanics_definitions outputs "door_hinge_field_results" field variables plastic_strain 0 equivalent
173 solid_mechanics_definitions outputs "door_pin_field_results" new
174 solid_mechanics_definitions outputs "door_pin_field_results" field database_name "door_pin_results"
175 solid_mechanics_definitions outputs "door_pin_field_results" field interval "simulation_output_interval"
176 solid_mechanics_definitions outputs "door_pin_field_results" field part "door_pin"
177 solid_mechanics_definitions outputs "door_pin_field_results" field variables displacement 0 x
178 solid_mechanics_definitions outputs "door_pin_field_results" field variables displacement 1 y
179 solid_mechanics_definitions outputs "door_pin_field_results" field variables displacement 2 z
180 solid_mechanics_definitions outputs "door_pin_field_results" field variables stress 0 von_mises
181 solid_mechanics_definitions outputs "door_pin_field_results" field variables plastic_strain 0 equivalent
182 solid_mechanics_definitions outputs "door_link_reaction_force_results" new
183 solid_mechanics_definitions outputs "door_link_reaction_force_results" history interval "simulation_output_interval"
184 solid_mechanics_definitions outputs "door_link_reaction_force_results" history probe_variables 0 door_link_reaction_force
185 solid_mechanics_definitions outputs "door_hinge_reaction_force_results" new
186 solid_mechanics_definitions outputs "door_hinge_reaction_force_results" history interval "simulation_output_interval"
187 solid_mechanics_definitions outputs "door_hinge_reaction_force_results" history probe_variables 0 door_hinge_reaction_force
188 solid_mechanics_definitions outputs "door_hinge_max_vm_results" new
189 solid_mechanics_definitions outputs "door_hinge_max_vm_results" history interval "simulation_output_interval"
190 solid_mechanics_definitions outputs "door_hinge_max_vm_results" history probe_variables 0 door_hinge_max_vm
191 #!=FIELD OUTPUT
192 #!=PROCEDURES
193 label "door_hinge_failure"
194 procedures "pull_door_hinge" solid_mechanics
195 procedures "pull_door_hinge" solid_mechanics interval "simulation_interval"
196 procedures "pull_door_hinge" solid_mechanics flex_model "flex_models_i"
197 procedures "pull_door_hinge" solid_mechanics time_stepping_method "implicit_dynamics_quasistatic"
198 procedures "pull_door_hinge" solid_mechanics boundary_conditions 0 "hold_door_link_bottom"
199 procedures "pull_door_hinge" solid_mechanics boundary_conditions 1 "pull_door_hinge_bolts"
200 procedures "pull_door_hinge" solid_mechanics interactions 0 "frictional_contact"
201 procedures "pull_door_hinge" solid_mechanics outputs 0 "door_link_field_results"

```

```

202 procedures "pull_door_hinge" solid_mechanics outputs 1 "door_hinge_field_results"
203 procedures "pull_door_hinge" solid_mechanics outputs 2 "door_pin_field_results"
204 procedures "pull_door_hinge" solid_mechanics outputs 3 "door_link_reaction_force_results"
205 procedures "pull_door_hinge" solid_mechanics outputs 4 "door_hinge_reaction_force_results"
206 procedures pull_door_hinge solid_mechanics outputs 5 door_hinge_max_vm_results
207 #!=PROCEDURES
208 #=Save the Coreform Flex project
209 save "door_lock_assembly.cf"
210 #!=Save the Coreform Flex project
211 #=SUBMIT TRIM JOB
212 # Check the trimmed mesh
213 job "job_door_hinge_trim" new
214 job "job_door_hinge_trim" trim processor_count 16
215 job "job_door_hinge_trim" trim trim_parts [door_link door_hinge door_pin]
216 job "job_door_hinge_trim" submit
217 job "job_door_hinge_trim" wait
218 #!=SUBMIT TRIM JOB
219 #=SUBMIT SIMULATION JOB
220 job "job_door_hinge_failure" new
221 job "job_door_hinge_failure" simulation processor_count 16
222 job "job_door_hinge_failure" submit
223 job "job_door_hinge_failure" wait
224 #!=SUBMIT SIMULATION JOB
225 #=VISUALIZE SIMULATION RESULTS
226 job "job_door_hinge_failure" render_iga timestep last scalars stress von_mises
227 #!=VISUALIZE SIMULATION RESULTS
228

```

C.2.2. job_door_hinge_failure.iga

```

1 {
2   "coreform_iga_version": "2025.8",
3   "data_output_file": "cf_iga_data_output_ip2.json",
4   "flex_models": [
5     {
6       "database_name": "geom",
7       "label": "flex_models_1",
8       "parts": [
9         {
10          "label": "door_hinge_instance",
11          "material": "aluminum",
12          "material_model": "neo_hookean_isotropic_plasticity",
13          "part": "door_hinge",
14          "quadrature_options": {
15            "optimization_lawson_hanson_solve_completion": true,
16            "optimization_method": "tet_pruning",
17            "optimization_residual_tolerance": 1.0000000000000002e-12,
18            "optimized_cut_quadrature_strategy": "reduce_from_uncut",
19            "quadrature": "QP1",
20            "tessellation_option": "tessellation_options_1"
21          }
22        },
23        {
24          "label": "door_link_instance",
25          "material": "aluminum",
26          "material_model": "neo_hookean_isotropic_plasticity",
27          "part": "door_link",
28          "quadrature_options": {
29            "optimization_lawson_hanson_solve_completion": true,
30            "optimization_method": "tet_pruning",
31            "optimization_residual_tolerance": 1.0000000000000002e-12,
32            "optimized_cut_quadrature_strategy": "reduce_from_uncut",
33            "quadrature": "QP1",
34            "tessellation_option": "tessellation_options_1"
35          }
36        },
37        {
38          "label": "door_pin_instance",
39          "material": "steel",
40          "material_model": "neo_hookean_isotropic_plasticity",
41          "part": "door_pin",
42          "quadrature_options": {
43            "optimization_lawson_hanson_solve_completion": true,
44            "optimization_method": "tet_pruning",
45            "optimization_residual_tolerance": 1.0000000000000002e-12,
46            "optimized_cut_quadrature_strategy": "reduce_from_uncut",
47            "quadrature": "QP1",
48            "tessellation_option": "tessellation_options_1"
49          }
50        }
51      ]
52    }
53  ]
54 }

```

```

51     ],
52     "small_cell_volume_ratio": 0
53   }
54 ],
55 "functions": [
56   {
57     "label": "aluminum_plastic_hardening",
58     "piecewise_linear": {
59       "abscissa": [0, 10],
60       "ordinate": [0, 100]
61     }
62   },
63   {
64     "label": "steel_plastic_hardening",
65     "piecewise_linear": {
66       "abscissa": [0, 10],
67       "ordinate": [0, 100]
68     }
69   },
70   {
71     "constant": {
72       "value": 1
73     },
74     "label": "constant_1"
75   },
76   {
77     "label": "linear_ramp",
78     "piecewise_linear": {
79       "abscissa": [0, 1],
80       "ordinate": [0, 1]
81     }
82   }
83 ],
84 "intervals": [
85   {
86     "label": "simulation_interval",
87     "start_time": 0,
88     "stop_time": 1,
89     "time_increment": 0.0005
90   },
91   {
92     "label": "simulation_output_interval",
93     "step_increment": 1,
94     "time_increment": 1,
95     "use_start_stop_from_interval": "simulation_interval"
96   }
97 ],
98 "label": "door_hinge_failure",
99 "linear_equation_solvers": [
100   {
101     "direct": {
102       "lu": {
103         "use_diagonal_scaling": false
104       }
105     },
106     "label": "direct_lu"
107   },
108   {
109     "direct": {
110       "multi_frontal": {
111         "use_diagonal_scaling": false
112       }
113     },
114     "label": "direct_multi_frontal"
115   }
116 ],
117 "materials": [
118   {
119     "label": "aluminum",
120     "mass_density": 2.6999999999999998e-9,
121     "neo_hookean_isotropic_plasticity": {
122       "isotropic_plasticity": {
123         "hardening_function": "aluminum_plastic_hardening",
124         "yield_stress": 322
125       },
126       "neo_hookean": {
127         "poissons_ratio": 0.3,
128         "youngs_modulus": 73800
129       }
130     }
131   },
132   {
133     "label": "steel",
134     "mass_density": 2.6999999999999998e-9,
135     "neo_hookean_isotropic_plasticity": {
136       "isotropic_plasticity": {
137         "hardening_function": "steel_plastic_hardening",
138         "yield_stress": 2000
139       },
140       "neo_hookean": {
141         "poissons_ratio": 0.29,

```

```

142     "youngs_modulus": 200000
143   }
144 }
145 ],
146 ],
147 "nonlinear_equation_solvers": [
148   {
149     "label": "newton_raphson",
150     "newton": {
151       "linear_equation_solver": "direct_multi_frontal",
152       "maximum_iterations": 20,
153       "target_relative_residual": 1e-8,
154       "use_line_search": false
155     }
156   }
157 ],
158 "procedures": [
159   {
160     "label": "pull_door_hinge",
161     "solid_mechanics": {
162       "boundary_conditions": [
163         "hold_door_link_bottom",
164         "pull_door_hinge_bolts"
165       ],
166       "flex_model": "flex_models_1",
167       "interactions": [
168         "frictional_contact"
169       ],
170       "interval": "simulation_interval",
171       "outputs": [
172         "door_link_field_results",
173         "door_hinge_field_results",
174         "door_pin_field_results",
175         "door_link_reaction_force_results",
176         "door_hinge_reaction_force_results"
177       ],
178       "time_stepping_method": "implicit_dynamics_quasistatic"
179     }
180   }
181 ],
182 "solid_mechanics_definitions": {
183   "boundary_conditions": [
184     {
185       "displacement": {
186         "components": ["x", "y", "z"],
187         "function": "constant_1",
188         "scale_factor": 0
189       },
190       "label": "hold_door_link_bottom",
191       "penalty": {
192         "dimensionless": true,
193         "value": 10
194       },
195       "quadrature_options": {
196         "optimization_lawson_hanson_solve_completion": true,
197         "optimization_method": "tet_pruning",
198         "optimization_residual_tolerance": 1.0000000000000002e-12,
199         "optimized_cut_quadrature_strategy": "reduce_from_uncut",
200         "quadrature": "QP1",
201         "tessellation_option": "tessellation_options_1"
202       },
203       "set": "door_link_bottom_surfaces"
204     },
205     {
206       "displacement": {
207         "components": ["x", "y", "z"],
208         "function": "linear_ramp",
209         "scale_factor": [0, 0, 9]
210       },
211       "label": "pull_door_hinge_bolts",
212       "penalty": {
213         "dimensionless": true,
214         "value": 10
215       },
216       "quadrature_options": {
217         "optimization_lawson_hanson_solve_completion": true,
218         "optimization_method": "tet_pruning",
219         "optimization_residual_tolerance": 1.0000000000000002e-12,
220         "optimized_cut_quadrature_strategy": "reduce_from_uncut",
221         "quadrature": "QP1",
222         "tessellation_option": "tessellation_options_1"
223       },
224       "set": "door_hinge_bolt_surfaces"
225     }
226   ],
227   "interaction_properties": [
228     {
229       "coulomb_friction": {
230         "coefficient_with_regularization": {
231           "friction_coefficient": 0.3,
232           "regularization_velocity": 0.1

```

```

233     }
234   },
235   "label": "friction"
236 }
237 ],
238 "interactions": [
239   {
240     "label": "frictional_contact",
241     "mechanical_contact": {
242       "activation_distance": {
243         "dimensionless": true,
244         "value": 0.6
245       },
246       "activation_zone_influential_radius_factor": 1,
247       "angle_scaling": {
248         "negative_dot_product_lower_limit": 0,
249         "negative_dot_product_upper_limit": 1,
250         "ramp_function_continuity": "C0",
251         "scaling_strategy": "signed_distance"
252       },
253       "approximate_tangent_iterations": 6,
254       "base_contact": {
255         "complement": true
256       },
257       "collision_search_filter": {
258         "angle_filter": {
259           "threshold_angle": 30
260         },
261         "area_filter": {
262           "threshold_area_ratio": 10
263         },
264         "spurious_self_contact_filter": {
265           "use": true
266         }
267       },
268       "effective_distance_smoothing": {
269         "distance_smoothing_factor": 0.125
270       },
271       "interaction_properties": "friction",
272       "penalty": {
273         "dimensionless": true,
274         "value": 10
275       },
276       "potential_cutoff_distance_factor": 0.5
277     },
278     "quadrature_options": {
279       "optimization_lawson_hanson_solve_completion": true,
280       "optimization_method": "tet_pruning",
281       "optimization_residual_tolerance": 1.0000000000000002e-12,
282       "optimized_cut_quadrature_strategy": "reduce_from_uncut",
283       "quadrature": "QP1",
284       "tessellation_option": "tessellation_options_1"
285     }
286   }
287 ],
288 "outputs": [
289   {
290     "field": {
291       "database_name": "door_link_results_0p6",
292       "element_variable_output_strategy": "interpolate",
293       "external_surfaces_only": false,
294       "include_integration_point_output": false,
295       "interval": "simulation_output_interval",
296       "part": "door_link",
297       "variables": {
298         "displacement": ["x", "y", "z"],
299         "plastic_strain": ["equivalent"],
300         "stress": ["von_mises"]
301       }
302     },
303     "label": "door_link_field_results"
304   },
305   {
306     "field": {
307       "database_name": "door_hinge_results_0p6",
308       "element_variable_output_strategy": "interpolate",
309       "external_surfaces_only": false,
310       "include_integration_point_output": false,
311       "interval": "simulation_output_interval",
312       "part": "door_hinge",
313       "variables": {
314         "displacement": ["x", "y", "z"],
315         "plastic_strain": ["equivalent"],
316         "stress": ["von_mises"]
317       }
318     },
319     "label": "door_hinge_field_results"
320   },
321   {
322     "field": {
323       "database_name": "door_pin_results_0p6",

```

```

324         "element_variable_output_strategy": "interpolate",
325         "external_surfaces_only": false,
326         "include_integration_point_output": false,
327         "interval": "simulation_output_interval",
328         "part": "door_pin",
329         "variables": {
330             "displacement": ["x", "y", "z"],
331             "plastic_strain": ["equivalent"],
332             "stress": ["von_mises"]
333         }
334     },
335     "label": "door_pin_field_results"
336 },
337 {
338     "history": {
339         "interval": "simulation_output_interval",
340         "probe_variables": ["door_link_reaction_force"]
341     },
342     "label": "door_link_reaction_force_results"
343 },
344 {
345     "history": {
346         "interval": "simulation_output_interval",
347         "probe_variables": ["door_hinge_reaction_force"]
348     },
349     "label": "door_hinge_reaction_force_results"
350 }
351 ],
352 "probes": [
353     {
354         "integrated_surface_quantity": {
355             "use_set_from_boundary_condition": "hold_door_link_bottom",
356             "variables": {
357                 "reaction_force": ["x", "y", "z"]
358             }
359         },
360         "label": "door_link_reaction_force"
361     },
362     {
363         "integrated_surface_quantity": {
364             "use_set_from_boundary_condition": "pull_door_hingeBolts",
365             "variables": {
366                 "reaction_force": ["x", "y", "z"]
367             }
368         },
369         "label": "door_hinge_reaction_force"
370     }
371 ],
372 },
373 "tessellation_options": [
374     {
375         "label": "tessellation_options_1",
376         "meshgems": {
377             "average_triangle_edge_subdivisions": 4
378         }
379     }
380 ],
381 "time_steppers": [
382     {
383         "generalized_alpha": {
384             "adaptivity": {
385                 "decrease_factor": 0.25,
386                 "increase_factor": 1.5,
387                 "maximum_time_step": 0.1,
388                 "minimum_time_step": 0.00001
389             },
390             "nonlinear_equation_solver": "newton_raphson",
391             "predictor": "constant_velocity",
392             "spectral_radius": 0
393         },
394         "label": "implicit_dynamics_quasistatic"
395     }
396 ]
397 }

```

C.2.3. read_force_vm.py

```

1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """

```

```

4 Created on Thu Aug 21 10:42:45 2025
5
6 @author: danajer
7 """
8
9 import pandas as pd
10 import matplotlib.pyplot as plt
11 import numpy as np
12 import seaborn as sns
13
14 plt.close('all')
15
16 from matplotlib import rc
17
18 def set_style_sns():
19     sns.set_context('paper')
20     sns.set(font = 'serif')
21     sns.set(font_scale = 1.1)
22
23     sns.set_style('white', {
24         'font.family': 'serif',
25         'font.serif': ['Time', 'Palatino', 'serif'],
26         'lines.markersize': 10
27     })
28
29 plt.rcParams.update({'font.size': 8})
30 set_style_sns()
31
32 plt.rcParams.update({'text.usetex': False})
33
34 files = ['with_probe/results_0p8/cf_iga_data_output.json',
35         'with_probe/results_1p0/cf_iga_data_output.json',
36         'with_probe/results_1p2/cf_iga_data_output.json']
37 mesh_sizes = ['0p8', '1p0', '1p2']
38 directs = ['x', 'y', 'z']
39
40
41 plt.figure(figsize = (10,8))
42
43 force_mag = []
44 vm_stresses = []
45 dofs = np.array([260247, 154746, 101739])
46 runtimes = []
47 for ii,file in enumerate(files):
48     data = pd.read_json(file, orient = 'index')
49     data_f = data['history']['pull_door_hinge']['door_hinge_reaction_force']
50     time = data_f['time']
51     force = np.stack([data_f['reaction_force']['x'], data_f['reaction_force']['y'], data_f['reaction_force']['z']])
52     vm_stress = data['history']['pull_door_hinge']['door_hinge_max_vm']['extremum']['stress']['von_mises']
53     vm_stresses.append(vm_stress[-1])
54     force_mag.append(np.sqrt(force[0,-1]**2 + force[1,-1]**2 + force[2,-1]**2))
55     runtimes.append(data['monitors']['pull_door_hinge']['timing']['procedure_solve'][0])
56     for jj in range(3):
57         plt.subplot(2,2,jj+1)
58         plt.plot(time, force[jj,:], label = mesh_sizes[ii])
59
60
61         if ii ==2:
62             plt.xlabel('Time, s')
63             plt.ylabel('door hinge reaction force, lbf')
64             plt.title(directs[jj])
65             plt.legend()
66             plt.grid('on')
67
68     plt.subplot(2,2,4)
69     plt.plot(time, vm_stress, label = mesh_sizes[ii])
70     if ii ==2:
71         plt.xlabel('Time, s')
72         plt.ylabel('door hinge von Mises, psi')
73         plt.title(directs[jj])
74         plt.legend()
75         plt.grid('on')
76
77
78 force_mag = np.array(force_mag)
79 vm_stresses = np.array(vm_stresses)
80 runtimes = np.array(runtimes)
81
82 plt.tight_layout()
83 plt.savefig('reaction_force_comparison.png')
84
85 plt.show()
86
87
88 plt.figure(figsize = (8,8))
89 plt.subplot(2,1,1)
90 plt.plot(dofs, force_mag, '-.', label = 'IGA')
91 plt.ylabel('Reaction Force Magnitude, lbf')
92 plt.xlabel('Number of DOFs')
93 plt.grid('on')
94 plt.legend()

```



```

35     pnames.append(pname)
36
37 for ii in range(len(names)):
38     flex.cmd('model_tree "geometry frames {} label" "frame-{}" undefined '.format(str(ii),str(ii)))
39
40 ## flex models
41
42 flex.cmd('flex_models "flex_models_1" new ')
43 flex.cmd('flex_models "flex_models_1" database_name "geom" ')
44 flex.cmd('flex_models "flex_models_1" small_cell_volume_ratio 0')
45
46 for ii in range(len(pnames)):
47     instance_name = "instance_" + pnames[ii]
48     flex.cmd('flex_models "flex_models_1" parts "{}" part "{}" '.format(instance_name, pnames[ii]))
49     flex.cmd('flex_models "flex_models_1" parts "{}" material "aluminum" '.format(instance_name))
50     flex.cmd('flex_models "flex_models_1" parts "{}" material_model elastic '.format(instance_name))
51
52 ## modal
53 flex.cmd('structural_modal_analysis_definitions modal_solution_methods "modal_solution_methods_1" new')
54 flex.cmd('structural_modal_analysis_definitions modal_solution_methods modal_solution_methods_1 frequency_extraction minimum_frequency 1')
55 flex.cmd('structural_modal_analysis_definitions modal_solution_methods modal_solution_methods_1 frequency_extraction maximum_frequency 5000')
56
57 ## boundary conditions
58
59 flex.cmd('set "set_bc" pick curve 5448')
60 flex.cmd('set "set_bc" pick curve 5434')
61
62 flex.cmd('structural_modal_analysis_definitions boundary_conditions "fixed" new ')
63 flex.cmd('structural_modal_analysis_definitions boundary_conditions "fixed" set "set_bc" ')
64 flex.cmd('structural_modal_analysis_definitions boundary_conditions fixed components new')
65 flex.cmd('structural_modal_analysis_definitions boundary_conditions fixed components 0 x')
66 flex.cmd('structural_modal_analysis_definitions boundary_conditions fixed components new')
67 flex.cmd('structural_modal_analysis_definitions boundary_conditions fixed components 1 y')
68 flex.cmd('structural_modal_analysis_definitions boundary_conditions fixed components new')
69 flex.cmd('structural_modal_analysis_definitions boundary_conditions fixed components 2 z')
70
71 ## output
72
73 flex.cmd('structural_modal_analysis_definitions outputs "outputs_1" new')
74 flex.cmd('structural_modal_analysis_definitions outputs outputs_1 field database_name "cb_modal_0p1"')
75 flex.cmd('intervals "intervals_1" new')
76 flex.cmd('structural_modal_analysis_definitions outputs outputs_1 field interval "intervals_1"')
77 flex.cmd('intervals intervals_1 start_time 1')
78 flex.cmd('intervals intervals_1 step_increment 1')
79 flex.cmd('structural_modal_analysis_definitions outputs outputs_1 field variables displacement new')
80 flex.cmd('structural_modal_analysis_definitions outputs outputs_1 field variables displacement 0 all')
81 flex.cmd('structural_modal_analysis_definitions outputs outputs_1 field part "part_1" ')
82
83
84 flex.cmd('set "all_parts" new ')
85 for ii in range(len(pnames)):
86     flex.cmd('set "all_parts" add parts [{}].format(ii + 1))
87
88 flex.cmd('structural_modal_analysis_definitions outputs "outputs_2" new')
89 flex.cmd('structural_modal_analysis_definitions outputs outputs_2 field database_name "cb_all_parts_modal_0p1"')
90 flex.cmd('intervals "intervals_1" new')
91 flex.cmd('structural_modal_analysis_definitions outputs outputs_2 field interval "intervals_1"')
92 flex.cmd('intervals intervals_1 start_time 1')
93 flex.cmd('intervals intervals_1 step_increment 1')
94 flex.cmd('structural_modal_analysis_definitions outputs outputs_2 field variables displacement new')
95 flex.cmd('structural_modal_analysis_definitions outputs outputs_2 field variables displacement 0 all')
96 flex.cmd('structural_modal_analysis_definitions outputs outputs_2 field set "all_parts" ')
97
98 ## functions
99
100 flex.cmd('functions "constant_1" new ')
101 flex.cmd('functions "constant_1" constant value 1 ')
102
103 ## procedure
104
105 flex.cmd('procedures procedures_1 structural_modal_analysis flex_model "flex_models_1" ')
106 flex.cmd('procedures procedures_1 structural_modal_analysis modal_solution_method "modal_solution_methods_1"')
107 flex.cmd('procedures procedures_1 structural_modal_analysis boundary_conditions new')
108 flex.cmd('procedures procedures_1 structural_modal_analysis boundary_conditions 0 fixed')
109 flex.cmd('procedures procedures_1 structural_modal_analysis interactions new')
110 flex.cmd('procedures procedures_1 structural_modal_analysis interactions 0 delete')
111 flex.cmd('procedures procedures_1 structural_modal_analysis outputs new')
112 flex.cmd('procedures procedures_1 structural_modal_analysis outputs 0 outputs_1')
113 flex.cmd('procedures procedures_1 structural_modal_analysis outputs new')
114 flex.cmd('procedures procedures_1 structural_modal_analysis outputs 1 outputs_2')
115
116 ## interactions
117
118 flex.cmd('procedures procedures_1 structural_modal_analysis interactions new')
119 flex.cmd('structural_modal_analysis_definitions interactions "interactions_1" new')
120 flex.cmd('procedures procedures_1 structural_modal_analysis interactions 0 interactions_1')
121 flex.cmd('structural_modal_analysis_definitions interactions interactions_1 tied_constraint active')
122 flex.cmd('structural_modal_analysis_definitions interaction_search_parameters "interaction_search_parameters_1" new')
123 flex.cmd('structural_modal_analysis_definitions interactions interactions_1 tied_constraint region_search_method
↵ "interaction_search_parameters_1"')
124 flex.cmd('structural_modal_analysis_definitions interactions interactions_1 tied_constraint target_set "all_parts" ')

```

```

125 flex.cmd('structural_modal_analysis_definitions interactions interactions_1 tied_constraint source_set "all_parts" ')
126
127 ## Check the trimmed mesh
128 flex.cmd('job "job_cb_modal_001" new ')
129 flex.cmd('job "job_cb_modal_001" trim processor_count 16 ')
130 flex.cmd('model_tree "job_manager queues local working_dir" "jobs_cb_modal_001" undefined')
131
132 fstr = '['
133 for ii in range(len(names)):
134     fstr = fstr + '{' + '{' + '.format(pnames[ii])
135
136 fstr = fstr + ']'
137
138 flex.cmd('job "job_cb_modal_001" trim trim_parts {} '.format(fstr))
139
140 flex.cmd('model_tree "job_manager jobs 0 job_type trim processor_count" "64" undefined')
141 flex.cmd('model_tree "job_manager jobs 0 job_type trim parts" new')
142 flex.cmd('model_tree "job_manager jobs 0 job_type trim parts 0" "part_1" undefined')
143 flex.cmd('model_tree "job_manager jobs 0 job_type simulation processor_count" "64" undefined')
144 flex.cmd('model_tree "job_manager jobs 0 job_type simulation render variable displacement" "undefined" undefined')
145 flex.cmd('model_tree "job_manager jobs 0 job_type simulation render variable displacement" "all" undefined')
146
147 ## end
148
149 ## intervals
150
151 flex.cmd('intervals "simulation_interval" new')
152 flex.cmd('intervals "simulation_interval" start_time 0')
153 flex.cmd('intervals intervals_1 start_step 1')
154 flex.cmd('intervals "simulation_interval" stop_time 1')
155 flex.cmd('intervals "simulation_interval" time_increment 5e-4')
156 flex.cmd('intervals "simulation_output_interval" new')
157 flex.cmd('intervals "simulation_output_interval" use_start_stop_from_interval "simulation_interval"')
158 flex.cmd('intervals "simulation_output_interval" step_increment 1')
159 #flex.cmd('intervals simulation_output_interval number_equally_spaced_values 10')
160
161 ## linear eq solvers
162
163 flex.cmd('linear_equation_solvers "direct_lu" new')
164 flex.cmd('linear_equation_solvers "direct_lu" direct lu')
165 flex.cmd('linear_equation_solvers "direct_multi_frontal" new')
166 flex.cmd('linear_equation_solvers "direct_multi_frontal" direct multi_frontal')
167
168 ## nonlinear eq. solvers
169
170 flex.cmd('nonlinear_equation_solvers "newton_raphson" new')
171 flex.cmd('nonlinear_equation_solvers "newton_raphson" newton linear_equation_solver "direct_multi_frontal"')
172 flex.cmd('nonlinear_equation_solvers "newton_raphson" newton maximum_iterations 20')
173 flex.cmd('nonlinear_equation_solvers "newton_raphson" newton target_relative_residual 1e-8')
174
175 ## timesteppers
176
177 flex.cmd('time_steppers "implicit_dynamics_quasistatic" new ')
178 flex.cmd('time_steppers "implicit_dynamics_quasistatic" generalized_alpha spectral_radius 0 ')
179 flex.cmd('time_steppers "implicit_dynamics_quasistatic" generalized_alpha predictor constant_velocity ')
180 flex.cmd('time_steppers "implicit_dynamics_quasistatic" generalized_alpha nonlinear_equation_solver "newton_raphson" ')
181 flex.cmd('time_steppers "implicit_dynamics_quasistatic" generalized_alpha adaptivity maximum_time_step 0.1 ')
182 flex.cmd('time_steppers "implicit_dynamics_quasistatic" generalized_alpha adaptivity minimum_time_step 1e-5 ')
183 flex.cmd('time_steppers "implicit_dynamics_quasistatic" generalized_alpha adaptivity decrease_factor 0.25 ')
184 flex.cmd('time_steppers "implicit_dynamics_quasistatic" generalized_alpha adaptivity increase_factor 1.5 ')
185
186 ## functions
187
188 flex.cmd('functions "constant_1" new ')
189 flex.cmd('functions "constant_1" constant value 1 ')
190 flex.cmd('functions "linear_ramp" new ')
191 flex.cmd('functions "linear_ramp" piecewise_linear abscissa [0 1] ')
192 flex.cmd('functions "linear_ramp" piecewise_linear ordinate [0 1] ')
193
194 ## boundary conditions
195
196 flex.cmd('set "set_bc" pick curve 5448')
197 flex.cmd('set "set_bc" pick curve 5434')
198
199 flex.cmd('solid_mechanics_definitions boundary_conditions "fixed" new ')
200 flex.cmd('solid_mechanics_definitions boundary_conditions "fixed" set "set_bc" ')
201 flex.cmd('solid_mechanics_definitions boundary_conditions "fixed" displacement function "constant_1" ')
202 flex.cmd('solid_mechanics_definitions boundary_conditions "fixed" displacement components 0 x ')
203 flex.cmd('solid_mechanics_definitions boundary_conditions "fixed" displacement components 1 y ')
204 flex.cmd('solid_mechanics_definitions boundary_conditions "fixed" displacement components 2 z ')
205 flex.cmd('solid_mechanics_definitions boundary_conditions "fixed" displacement scale_factor 0 ')
206
207 ## load
208
209 flex.cmd('solid_mechanics_definitions load_conditions "load_conditions_1" new')
210 flex.cmd('solid_mechanics_definitions load_conditions load_conditions_1 body_force active ')
211 flex.cmd('solid_mechanics_definitions load_conditions load_conditions_1 body_force components new ')
212 flex.cmd('solid_mechanics_definitions load_conditions load_conditions_1 body_force components 0 y ')
213 flex.cmd('solid_mechanics_definitions load_conditions load_conditions_1 body_force function "linear_ramp" ')
214 flex.cmd('solid_mechanics_definitions load_conditions load_conditions_1 part "part_1" ')
215 flex.cmd('solid_mechanics_definitions load_conditions load_conditions_1 body_force scale_factor [50] ')

```

```

216
217 ## output
218
219 flex.cmd('solid_mechanics_definitions outputs "cb_statics_results" new ')
220 flex.cmd('solid_mechanics_definitions outputs "cb_statics_results" field database_name "cb_statics_result"')
221 flex.cmd('solid_mechanics_definitions outputs "cb_statics_results" field interval "simulation_output_interval"')
222 flex.cmd('solid_mechanics_definitions outputs "cb_statics_results" field part "part_1" ')
223 flex.cmd('solid_mechanics_definitions outputs "cb_statics_results" field variables displacement 0 x ')
224 flex.cmd('solid_mechanics_definitions outputs "cb_statics_results" field variables displacement 1 y ')
225 flex.cmd('solid_mechanics_definitions outputs "cb_statics_results" field variables displacement 2 z ')
226 flex.cmd('solid_mechanics_definitions outputs "cb_statics_results" field variables stress 0 von_mises ')
227
228 ## procedure
229
230 flex.cmd('label "uur_mech_analysis" ')
231 flex.cmd('procedures "cb_statics" solid_mechanics ')
232 flex.cmd('procedures "cb_statics" solid_mechanics interval "simulation_interval" ')
233 flex.cmd('procedures "cb_statics" solid_mechanics flex_model "flex_models_1" ')
234 flex.cmd('procedures "cb_statics" solid_mechanics time_stepping_method "implicit_dynamics_quasistatic" ')
235 flex.cmd('procedures "cb_statics" solid_mechanics boundary_conditions 0 "fixed" ')
236
237 flex.cmd('procedures "cb_statics" solid_mechanics outputs 0 "cb_statics_results" ')
238 flex.cmd('procedures "cb_statics" solid_mechanics load_conditions 0 load_conditions_1 ')
239
240 flex.cmd('model_tree "job_manager jobs 0 label" "cb_static_" undefined')
241 flex.cmd('model_tree "job_manager jobs 0 label" "cb_static_0p1" undefined')
242 flex.cmd('model_tree "job_manager jobs 0 job_type trim" "undefined" undefined')
243 flex.cmd('model_tree "job_manager jobs 0 job_type trim parts" new')
244 flex.cmd('model_tree "job_manager jobs 0 job_type trim parts 0" "part_1" undefined')
245 flex.cmd('model_tree "job_manager jobs 0 job_type trim processor_count" "24" undefined')

```

C.3.2. cb_modal_workflow_journal.py

```

1 #!python
2
3 import pickle
4
5 flex.cmd('reset')
6 flex.cmd('cd "C:\\Users\\danajer\\Documents\\fy25_coreform_eval\\iga\\circuit_board" ')
7
8 flex.cmd('open "C:\\Users\\danajer\\Documents\\fy25_coreform_eval\\iga\\circuit_board\\united_cb.cf" ')
9
10 flex.cmd('materials "materials_1" new')
11 flex.cmd('materials materials_1 label "aluminum" ')
12 flex.cmd('materials aluminum mass_density 0.0002538597037 ')
13 flex.cmd('materials aluminum elastic youngs_modulus 9.9e6')
14 flex.cmd('materials aluminum elastic poisson_ratio 0.33')
15 flex.cmd('materials aluminum elastic thermal_expansion thermal_expansion_coefficient 1e-6 ')
16 flex.cmd('materials aluminum elastic thermal_expansion reference_temperature 0.0 ') # assume C
17
18 names = ['ARDUINO_UNO_REV3_SMD_NAU01-1001-003-02000-1001-003-02000-Chamfer3']
19 char_lens = [0.3]
20
21 flex.cmd('data_output_file "cf_iga_data_output_0p1.json"')
22 flex.cmd('label "cb_modal_0p1"')
23
24 pnames = []
25 for ii in range(len(names)):
26     pname = "part_{}".format(str(ii + 1))
27     flex.cmd('name part {} "{}" '.format(ii + 1, pname))
28     mesh_name = "mesh_" + pname #names[ii]
29     ms = char_lens[ii]/3.0
30     print('mesh "{}" rectilinear degree 2 continuity 1 element_size [{} {} {}] padding [2 2 2]'.format(mesh_name, ms, ms, ms))
31     flex.cmd('mesh "{}" rectilinear degree 2 continuity 1 element_size [{} {} {}] padding [2 2 2]'.format(mesh_name, ms, ms, ms))
32     flex.cmd('part {} mesh "{}" '.format(pname, mesh_name))
33     flex.cmd('part {} volume_box'.format(pname))
34     flex.cmd('visibility mesh [{}] toggle '.format(mesh_name))
35     pnames.append(pname)
36
37 for ii in range(len(names)):
38     flex.cmd('model_tree "geometry frames {} label" "frame-{}" undefined '.format(str(ii),str(ii)))
39
40 ## flex models
41
42 flex.cmd('flex_models "flex_models_1" new ')
43 flex.cmd('flex_models "flex_models_1" database_name "geom" ')
44 flex.cmd('flex_models "flex_models_1" small_cell_volume_ratio 0')
45
46 for ii in range(len(pnames)):
47     instance_name = "instance_" + pnames[ii]

```

```

48 flex.cmd('flex_models "flex_models_1" parts "{}" part "{}" '.format(instance_name, pnames[i]))
49 flex.cmd('flex_models "flex_models_1" parts "{}" material "aluminum" '.format(instance_name))
50 flex.cmd('flex_models "flex_models_1" parts "{}" material_model elastic '.format(instance_name))
51
52 ## modal
53 flex.cmd('structural_modal_analysis_definitions modal_solution_methods "modal_solution_methods_1" new')
54 flex.cmd('structural_modal_analysis_definitions modal_solution_methods modal_solution_methods_1 frequency_extraction minimum_frequency 1')
55 flex.cmd('structural_modal_analysis_definitions modal_solution_methods modal_solution_methods_1 frequency_extraction maximum_frequency 5000')
56
57 ## boundary conditions
58
59 flex.cmd('set "set_bc" pick curve 5448')
60 flex.cmd('set "set_bc" pick curve 5434')
61
62 flex.cmd('structural_modal_analysis_definitions boundary_conditions "fixed" new ')
63 flex.cmd('structural_modal_analysis_definitions boundary_conditions "fixed" set "set_bc" ')
64 flex.cmd('structural_modal_analysis_definitions boundary_conditions fixed components new')
65 flex.cmd('structural_modal_analysis_definitions boundary_conditions fixed components 0 x')
66 flex.cmd('structural_modal_analysis_definitions boundary_conditions fixed components new')
67 flex.cmd('structural_modal_analysis_definitions boundary_conditions fixed components 1 y')
68 flex.cmd('structural_modal_analysis_definitions boundary_conditions fixed components new')
69 flex.cmd('structural_modal_analysis_definitions boundary_conditions fixed components 2 z')
70
71 ## output
72
73 flex.cmd('structural_modal_analysis_definitions outputs "outputs_1" new')
74 flex.cmd('structural_modal_analysis_definitions outputs outputs_1 field database_name "cb_modal_0p1"')
75 flex.cmd('intervals "intervals_1" new')
76 flex.cmd('structural_modal_analysis_definitions outputs outputs_1 field interval "intervals_1"')
77 flex.cmd('intervals intervals_1 start_time 1')
78 flex.cmd('intervals intervals_1 step_increment 1')
79 flex.cmd('structural_modal_analysis_definitions outputs outputs_1 field variables displacement new')
80 flex.cmd('structural_modal_analysis_definitions outputs outputs_1 field variables displacement 0 all')
81 flex.cmd('structural_modal_analysis_definitions outputs outputs_1 field part "part_1" ')
82
83 ## functions
84
85 flex.cmd('functions "constant_1" new ')
86 flex.cmd('functions "constant_1" constant value 1 ')
87
88 ## procedure
89
90 flex.cmd('procedures procedures_1 structural_modal_analysis flex_model "flex_models_1" ')
91 flex.cmd('procedures procedures_1 structural_modal_analysis modal_solution_method "modal_solution_methods_1"')
92 flex.cmd('procedures procedures_1 structural_modal_analysis boundary_conditions new')
93 flex.cmd('procedures procedures_1 structural_modal_analysis boundary_conditions 0 fixed')
94 flex.cmd('procedures procedures_1 structural_modal_analysis interactions new')
95 flex.cmd('procedures procedures_1 structural_modal_analysis interactions 0 delete')
96 flex.cmd('procedures procedures_1 structural_modal_analysis outputs new')
97 flex.cmd('procedures procedures_1 structural_modal_analysis outputs 0 outputs_1')
98
99 flex.cmd('model_tree "job_manager jobs 0 job_type trim processor_count" "64" undefined')
100 flex.cmd('model_tree "job_manager jobs 0 job_type trim parts" new')
101 flex.cmd('model_tree "job_manager jobs 0 job_type trim parts 0" "part_1" undefined')
102 flex.cmd('model_tree "job_manager jobs 0 job_type simulation processor_count" "64" undefined')
103 flex.cmd('model_tree "job_manager jobs 0 job_type simulation render variable displacement" "undefined" undefined')
104 flex.cmd('model_tree "job_manager jobs 0 job_type simulation render variable displacement" "all" undefined')
105
106 ## end
107
108 ## intervals
109
110 flex.cmd('intervals "simulation_interval" new')
111 flex.cmd('intervals "simulation_interval" start_time 0')
112 flex.cmd('intervals "simulation_interval" stop_time 1')
113 flex.cmd('intervals "simulation_interval" time_increment 5e-4')
114 flex.cmd('intervals "simulation_output_interval" new')
115 flex.cmd('intervals "simulation_output_interval" use_start_stop_from_interval "simulation_interval"')
116 flex.cmd('intervals "simulation_output_interval" step_increment 1')
117 #flex.cmd('intervals simulation_output_interval number_equally_spaced_values 10')
118
119 ## linear eq solvers
120
121 flex.cmd('linear_equation_solvers "direct_lu" new')
122 flex.cmd('linear_equation_solvers "direct_lu" direct lu')
123 flex.cmd('linear_equation_solvers "direct_multi_frontal" new')
124 flex.cmd('linear_equation_solvers "direct_multi_frontal" direct multi_frontal')
125
126 ## nonlinear eq. solvers
127
128 flex.cmd('nonlinear_equation_solvers "newton_raphson" new')
129 flex.cmd('nonlinear_equation_solvers "newton_raphson" newton linear_equation_solver "direct_multi_frontal"')
130 flex.cmd('nonlinear_equation_solvers "newton_raphson" newton maximum_iterations 20')
131 flex.cmd('nonlinear_equation_solvers "newton_raphson" newton target_relative_residual 1e-8')
132
133 ## timesteppers
134
135 flex.cmd('time_steppers "implicit_dynamics_quasistatic" new ')
136 flex.cmd('time_steppers "implicit_dynamics_quasistatic" generalized_alpha spectral_radius 0 ')
137 flex.cmd('time_steppers "implicit_dynamics_quasistatic" generalized_alpha predictor constant_velocity ')
138 flex.cmd('time_steppers "implicit_dynamics_quasistatic" generalized_alpha nonlinear_equation_solver "newton_raphson" ')

```

```

139 flex.cmd('time_steppers "implicit_dynamics_quasistatic" generalized_alpha adaptivity maximum_time_step 0.1 ')
140 flex.cmd('time_steppers "implicit_dynamics_quasistatic" generalized_alpha adaptivity minimum_time_step 1e-5 ')
141 flex.cmd('time_steppers "implicit_dynamics_quasistatic" generalized_alpha adaptivity decrease_factor 0.25 ')
142 flex.cmd('time_steppers "implicit_dynamics_quasistatic" generalized_alpha adaptivity increase_factor 1.5 ')
143
144 ## functions
145
146 flex.cmd('functions "constant_1" new ')
147 flex.cmd('functions "constant_1" constant value 1 ')
148 flex.cmd('functions "linear_ramp" new ')
149 flex.cmd('functions "linear_ramp" piecewise_linear abscissa [0 1] ')
150 flex.cmd('functions "linear_ramp" piecewise_linear ordinate [0 1] ')
151
152 ## boundary conditions
153
154 flex.cmd('set "set_bc" pick curve 5448')
155 flex.cmd('set "set_bc" pick curve 5434')
156
157 flex.cmd('solid_mechanics_definitions boundary_conditions "fixed" new ')
158 flex.cmd('solid_mechanics_definitions boundary_conditions "fixed" set "set_bc" ')
159 flex.cmd('solid_mechanics_definitions boundary_conditions "fixed" displacement function "constant_1" ')
160 flex.cmd('solid_mechanics_definitions boundary_conditions "fixed" displacement components 0 x ')
161 flex.cmd('solid_mechanics_definitions boundary_conditions "fixed" displacement components 1 y ')
162 flex.cmd('solid_mechanics_definitions boundary_conditions "fixed" displacement components 2 z ')
163 flex.cmd('solid_mechanics_definitions boundary_conditions "fixed" displacement scale_factor 0 ')
164
165 ## load
166
167 flex.cmd('solid_mechanics_definitions load_conditions "load_conditions_1" new')
168 flex.cmd('solid_mechanics_definitions load_conditions load_conditions_1 body_force active ')
169 flex.cmd('solid_mechanics_definitions load_conditions load_conditions_1 body_force components new ')
170 flex.cmd('solid_mechanics_definitions load_conditions load_conditions_1 body_force components 0 y ')
171 flex.cmd('solid_mechanics_definitions load_conditions load_conditions_1 body_force function "linear_ramp" ')
172 flex.cmd('solid_mechanics_definitions load_conditions load_conditions_1 part "part_1" ')
173 flex.cmd('solid_mechanics_definitions load_conditions load_conditions_1 body_force scale_factor [50] ')
174
175 ## output
176
177 flex.cmd('solid_mechanics_definitions outputs "cb_statics_results" new ')
178 flex.cmd('solid_mechanics_definitions outputs "cb_statics_results" field database_name "cb_statics_result"')
179 flex.cmd('solid_mechanics_definitions outputs "cb_statics_results" field interval "simulation_output_interval"')
180 flex.cmd('solid_mechanics_definitions outputs "cb_statics_results" field part "part_1" ')
181 flex.cmd('solid_mechanics_definitions outputs "cb_statics_results" field variables displacement 0 x ')
182 flex.cmd('solid_mechanics_definitions outputs "cb_statics_results" field variables displacement 1 y ')
183 flex.cmd('solid_mechanics_definitions outputs "cb_statics_results" field variables displacement 2 z ')
184 flex.cmd('solid_mechanics_definitions outputs "cb_statics_results" field variables stress 0 von_mises ')
185
186 ## procedure
187
188 flex.cmd('label "uur_mech_analysis" ')
189 flex.cmd('procedures "cb_statics" solid_mechanics ')
190 flex.cmd('procedures "cb_statics" solid_mechanics interval "simulation_interval" ')
191 flex.cmd('procedures "cb_statics" solid_mechanics flex_model "flex_models_1" ')
192 flex.cmd('procedures "cb_statics" solid_mechanics time_stepping_method "implicit_dynamics_quasistatic" ')
193 flex.cmd('procedures "cb_statics" solid_mechanics boundary_conditions 0 "fixed" ')
194
195 flex.cmd('procedures "cb_statics" solid_mechanics outputs 0 "cb_statics_results" ')
196 flex.cmd('procedures "cb_statics" solid_mechanics load_conditions 0 load_conditions_1 ')
197
198 flex.cmd('model_tree "job_manager jobs 0 label" "cb_static-" undefined')
199 flex.cmd('model_tree "job_manager jobs 0 label" "cb_static_0p1" undefined')
200 flex.cmd('model_tree "job_manager jobs 0 job_type trim" "undefined" undefined')
201 flex.cmd('model_tree "job_manager jobs 0 job_type trim parts" new')
202 flex.cmd('model_tree "job_manager jobs 0 job_type trim parts 0" "part_1" undefined')
203 flex.cmd('model_tree "job_manager jobs 0 job_type trim processor_count" "24" undefined')

```

C.3.3. cb_workflow_journal.py

```

1 #!python
2
3 import pickle
4
5 flex.cmd('reset')
6 flex.cmd('cd "C:\\Users\\danajer\\Documents\\fy25_coreform_eval\\iga\\circuit_board" ')
7
8 flex.cmd('open "C:\\Users\\danajer\\Documents\\fy25_coreform_eval\\iga\\circuit_board\\united.cb.cf" ')
9
10 flex.cmd('materials "materials_1" new')
11 flex.cmd('materials materials_1 label "aluminum" ')
12 flex.cmd('materials aluminum mass_density 0.0002538597037 ')

```

```

13 flex.cmd('materials aluminum elastic youngs_modulus 9.9e6')
14 flex.cmd('materials aluminum elastic poissons_ratio 0.33')
15 flex.cmd('materials aluminum elastic thermal_expansion thermal_expansion_coefficient 1e-6 ')
16 flex.cmd('materials aluminum elastic thermal_expansion reference_temperature 0.0 ') # assume C
17
18 names = ['ARDUINO_UNO_REV3_SMD_NAU01-1001-003-02000-1001-003-02000-Chamfer3']
19 char_lens = [0.3]
20
21 flex.cmd('data_output_file "cf_iga_data_output_0p1.json"')
22 flex.cmd('label "cb_static_grav_0p1"')
23
24 pnames = []
25 for ii in range(len(names)):
26     pname = "part_{}".format(str(ii + 1))
27     flex.cmd(' name part {} "{}" '.format(ii + 1, pname))
28     mesh_name = "mesh_" + pname #names[ii]
29     ms = char_lens[ii]/3.0
30     print('mesh "{}" rectilinear degree 2 continuity 1 element_size [{} {} {}] padding [2 2 2]'.format(mesh_name, ms, ms, ms))
31     flex.cmd('mesh "{}" rectilinear degree 2 continuity 1 element_size [{} {} {}] padding [2 2 2]'.format(mesh_name, ms, ms, ms))
32     flex.cmd('part "{}" mesh "{}" '.format(pname, mesh_name))
33     flex.cmd('part "{}" volume_box'.format(pname))
34     flex.cmd(' visibility mesh [{}] toggle '.format(mesh_name))
35     pnames.append(pname)
36
37 for ii in range(len(names)):
38     flex.cmd('model_trees "geometry frames {} label" "frame-{}" undefined '.format(str(ii),str(ii)))
39
40 ## flex models
41
42 flex.cmd('flex_models "flex_models_1" new ')
43 flex.cmd('flex_models "flex_models_1" database_name "geom" ')
44 flex.cmd('flex_models "flex_models_1" small_cell_volume_ratio 0')
45
46 for ii in range(len(pnames)):
47     instance_name = "instance_" + pnames[ii]
48     flex.cmd('flex_models "flex_models_1" parts "{}" part "{}" '.format(instance_name, pnames[ii]))
49     flex.cmd('flex_models "flex_models_1" parts "{}" material "aluminum" '.format(instance_name))
50     flex.cmd('flex_models "flex_models_1" parts "{}" material_model elastic '.format(instance_name))
51
52 ## intervals
53
54 flex.cmd('intervals "simulation_interval" new')
55 flex.cmd('intervals "simulation_interval" start_time 0')
56 flex.cmd('intervals "simulation_interval" stop_time 1')
57 flex.cmd('intervals "simulation_interval" time_increment 5e-4')
58 flex.cmd('intervals "simulation_output_interval" new')
59 flex.cmd('intervals "simulation_output_interval" use_start_stop_from_interval "simulation_interval"')
60 #flex.cmd('intervals "simulation_output_interval" step_increment 1')
61 flex.cmd('intervals simulation_output_interval number_equally_spaced_values 10')
62
63 ## linear eq solvers
64
65 flex.cmd('linear_equation_solvers "direct_lu" new')
66 flex.cmd('linear_equation_solvers "direct_lu" direct lu')
67 flex.cmd('linear_equation_solvers "direct_multi_frontal" new')
68 flex.cmd('linear_equation_solvers "direct_multi_frontal" direct multi_frontal')
69
70 ## nonlinear eq. solvers
71
72 flex.cmd('nonlinear_equation_solvers "newton_raphson" new')
73 flex.cmd('nonlinear_equation_solvers "newton_raphson" newton linear_equation_solver "direct_multi_frontal"')
74 flex.cmd('nonlinear_equation_solvers "newton_raphson" newton maximum_iterations 20')
75 flex.cmd('nonlinear_equation_solvers "newton_raphson" newton target_relative_residual 1e-8')
76
77 ## timesteppers
78
79 flex.cmd('time_steppers "implicit_dynamics_quasistatic" new ')
80 flex.cmd('time_steppers "implicit_dynamics_quasistatic" generalized_alpha spectral_radius 0 ')
81 flex.cmd('time_steppers "implicit_dynamics_quasistatic" generalized_alpha predictor constant_velocity ')
82 flex.cmd('time_steppers "implicit_dynamics_quasistatic" generalized_alpha nonlinear_equation_solver "newton_raphson" ')
83 flex.cmd('time_steppers "implicit_dynamics_quasistatic" generalized_alpha adaptivity maximum_time_step 0.1 ')
84 flex.cmd('time_steppers "implicit_dynamics_quasistatic" generalized_alpha adaptivity minimum_time_step 1e-5 ')
85 flex.cmd('time_steppers "implicit_dynamics_quasistatic" generalized_alpha adaptivity decrease_factor 0.25 ')
86 flex.cmd('time_steppers "implicit_dynamics_quasistatic" generalized_alpha adaptivity increase_factor 1.5 ')
87
88 ## functions
89
90 flex.cmd('functions "constant_1" new ')
91 flex.cmd('functions "constant_1" constant value 1 ')
92 flex.cmd('functions "linear_ramp" new ')
93 flex.cmd('functions "linear_ramp" piecewise_linear abscissa [0 1] ')
94 flex.cmd('functions "linear_ramp" piecewise_linear ordinate [0 1] ')
95
96 ## boundary conditions
97
98 flex.cmd('set "set_bc" pick curve 5448')
99 flex.cmd('set "set_bc" pick curve 5434')
100
101 flex.cmd('solid_mechanics_definitions boundary_conditions "fixed" new ')
102 flex.cmd('solid_mechanics_definitions boundary_conditions "fixed" set "set_bc" ')
103 flex.cmd('solid_mechanics_definitions boundary_conditions "fixed" displacement function "constant_1" ')

```

```

104 flex.cmd('solid_mechanics_definitions boundary_conditions "fixed" displacement components 0 x ')
105 flex.cmd('solid_mechanics_definitions boundary_conditions "fixed" displacement components 1 y ')
106 flex.cmd('solid_mechanics_definitions boundary_conditions "fixed" displacement components 2 z ')
107 flex.cmd('solid_mechanics_definitions boundary_conditions "fixed" displacement scale_factor 0 ')
108
109
110 ## load
111
112 flex.cmd('solid_mechanics_definitions load_conditions "load_conditions_1" new')
113 flex.cmd('solid_mechanics_definitions load_conditions load_conditions_1 body_force active ')
114 flex.cmd('solid_mechanics_definitions load_conditions load_conditions_1 body_force components new ')
115 flex.cmd('solid_mechanics_definitions load_conditions load_conditions_1 body_force components 0 y ')
116 flex.cmd('solid_mechanics_definitions load_conditions load_conditions_1 body_force function "linear_ramp" ')
117 flex.cmd('solid_mechanics_definitions load_conditions load_conditions_1 part "part_1" ')
118 flex.cmd('solid_mechanics_definitions load_conditions load_conditions_1 body_force scale_factor [50] ')
119
120 ## output
121
122 flex.cmd('solid_mechanics_definitions outputs "cb_statics_results" new ')
123 flex.cmd('solid_mechanics_definitions outputs "cb_statics_results" field database_name "cb_statics_result"')
124 flex.cmd('solid_mechanics_definitions outputs "cb_statics_results" field interval "simulation_output_interval"')
125 flex.cmd('solid_mechanics_definitions outputs "cb_statics_results" field part "part_1" ')
126 flex.cmd('solid_mechanics_definitions outputs "cb_statics_results" field variables displacement 0 x ')
127 flex.cmd('solid_mechanics_definitions outputs "cb_statics_results" field variables displacement 1 y ')
128 flex.cmd('solid_mechanics_definitions outputs "cb_statics_results" field variables displacement 2 z ')
129 flex.cmd('solid_mechanics_definitions outputs "cb_statics_results" field variables stress 0 von_mises ')
130
131 ## procedure
132
133 flex.cmd('label "uur_mech_analysis" ')
134 flex.cmd('procedures "cb_statics" solid_mechanics ')
135 flex.cmd('procedures "cb_statics" solid_mechanics interval "simulation_interval" ')
136 flex.cmd('procedures "cb_statics" solid_mechanics flex_model "flex_models_1" ')
137 flex.cmd('procedures "cb_statics" solid_mechanics time_stepping_method "implicit_dynamics_quasistatic" ')
138 flex.cmd('procedures "cb_statics" solid_mechanics boundary_conditions 0 "fixed" ')
139
140 flex.cmd('procedures "cb_statics" solid_mechanics outputs 0 "cb_statics_results" ')
141 flex.cmd('procedures "cb_statics" solid_mechanics load_conditions 0 load_conditions_1 ')
142
143 flex.cmd('model_tree "job_manager jobs 0 label" "cb_static_" undefined')
144 flex.cmd('model_tree "job_manager jobs 0 label" "cb_static_0p1" undefined')
145 flex.cmd('model_tree "job_manager jobs 0 job_type trim" "undefined" undefined')
146 flex.cmd('model_tree "job_manager jobs 0 job_type trim parts" new')
147 flex.cmd('model_tree "job_manager jobs 0 job_type trim parts 0" "part_1" undefined')
148 flex.cmd('model_tree "job_manager jobs 0 job_type trim processor_count" "24" undefined')

```



Sandia
National
Laboratories

Sandia National Laboratories
is a multimission laboratory
managed and operated by
National Technology &
Engineering Solutions of
Sandia LLC, a wholly owned
subsidiary of Honeywell
International Inc., for the U.S.
Department of Energy's
National Nuclear Security
Administration under contract
DE-NA0003525.